DOST-PCAARRD eLibrary

The smart-eLibrary Database

Structure, Schema, Semantics and Integrity

A technical reference and analytical study of the relational data layer

---

Version:	1.0
Date:	15 October 2026
Prepared by:	Joe Cris Viar, Developer
Schema state:	18 migrations, last 2026_10_13; MySQL 8.0.46
Evidence base:	73 database objects (62 tables, 11 views), 763 columns, application source code
Classification:	Internal technical documentation. Contains no personal data.

Abstract

This document describes and analyses the relational database behind the smart-eLibrary, the online public access catalogue (OPAC) and library back office of the DOST-PCAARRD eLibrary. It serves two readers at once. For reviewers it frames the database as an object of study: how it was examined, what conceptual model it realises, and how well it satisfies established principles of relational design. For developers and maintainers it provides a complete reference: every table, view, column, index, status code and the application events that create and change data.

The study combines three sources of evidence: introspection of a database built solely from the project's migrations (73 objects, 763 columns), confirmed to be identical to the operational schema; static analysis of the Laravel application code to recover meaning the schema does not express; and aggregate profiling of the operational data, collected without reading any personal records. The schema is organised into eight subject areas centred on two hubs, the bibliographic record (`catalogs`) and the account (`users`). Its most distinctive design decision is to store each MARC 21 record as a JSON document inside a relational row and to maintain a separate flattened projection (`view_catalogs`) as the read model for search and display.

The analysis finds a schema that is functionally coherent and whose current data are clean (no orphaned references), but whose integrity rests almost entirely on application code: only 6 of more than 30 relationships are declared as foreign keys, the same identifier is stored with seven different physical types, monetary amounts and dates are stored as text, several multi-step business operations are not atomic, and 18 objects belong to superseded designs, some of which are still referenced by live views. Twenty findings are reported with their evidence and consequences, and a prioritised programme of sixteen recommendations is proposed.

Keywords: *relational database design, entity-relationship modelling, normalisation, referential integrity, MARC 21, library management systems, Laravel, MySQL, data privacy.*

Contents

1 Introduction **5**
1.1 Purpose and scope 5
1.2 Readers and how to use this document 5
1.3 Conventions 5

2 Methodology **6**
2.1 Research questions 6
2.2 Evidence and procedure 6
2.3 Analytical framework 6
2.4 Threats to validity 7

3 Architecture of the data layer **8**
3.1 Platform and context 8
3.2 Physical characteristics 8
3.3 Provenance and evolution 9

4 Conceptual and logical model **10**
4.1 Subject areas 10
4.2 Bibliographic description and cataloguing 11
4.3 Holdings and circulation 12
4.4 People, organisations and access control 13
4.5 Engagement and discovery 14
4.6 Communication and feedback 15
4.7 Integration and configuration 16
4.8 Legacy structures 17

5 Data semantics: what the schema does not say **18**
5.1 The MARC 21 record in catalogs.field_data 18
5.2 The search projection 18
5.3 Status vocabularies and flags 19
5.4 The circulation lifecycle as a state machine 19
5.5 Access-control model 20

6 Event model: how actions become data **21**
6.1 Patron engagement 22
6.2 Catalogue propagation 22

7 Analysis **24**
7.1 Entity and referential integrity 24
7.2 Domain integrity 25
7.3 Transactional integrity and concurrency 26
7.4 Consistency between schema and code 27
7.5 Physical design and performance 27
7.6 Normalisation 28
7.7 Accountability and maintainability 29
7.8 Security and privacy 29

8 Recommendations **31**

9 Conclusion **33**

References **34**

Appendix A Data dictionary **35**

A.1 Bibliographic description and cataloguing....35
A.2 Holdings and digital objects....41
A.3 Circulation....44
A.4 People, organisations and access control....47
A.5 Engagement and discovery....51
A.6 Communication and feedback....54
A.7 Integration, configuration and framework....58
A.8 Legacy (pre-MARC) structures....59

Appendix B Index catalogue....63

Appendix C SQL view definitions....65

Appendix D Migration history....68

Appendix E Reference vocabularies....69

Appendix F Reproducing the evidence....70

1 Introduction

1.1 Purpose and scope

The smart-eLibrary stores its catalogue, holdings, circulation, patron accounts, engagement history and integration state in a single MySQL database. Much of what that database *means* is not visible in its schema: status codes are bare integers, a complete MARC 21 record lives inside one text column, and most relationships between tables exist only in the application's code. This document makes that meaning explicit and assesses the design against relational-database principles.

The scope is the database `elibrarydb` as defined by the project's migrations up to `2026_10_13_000000_settings_and_ai_sync_monitor_permissions`. It covers all 73 objects (62 base tables and 11 SQL views), their columns, keys, indexes and view definitions, the reference data that gives codes their meaning, and the application events that read and write each table. External stores that hold copies of database content (the Meilisearch index, the LanceDB index of the RAG worker and the file system holding PDFs and covers) are described only at their interface with the database.

1.2 Readers and how to use this document

Part I (chapters 1 to 9) is the study. Chapter 2 explains how the evidence was gathered and analysed. Chapters 3 to 6 describe the database at increasing levels of detail: architecture, conceptual and logical model, data semantics, and the event model. Chapter 7 analyses the design; chapter 8 turns the findings into recommendations; chapter 9 concludes.

Part II (appendices A to F) is the reference: a data dictionary for every object, the index catalogue, the SQL of every view, the migration history, the reference vocabularies, and the queries that reproduce the evidence. A developer looking up a single table can go straight to Appendix A.

1.3 Conventions

- `monospace` names a table, column, file or code identifier. Table and column names are given exactly as they appear in the schema, including misspellings such as `barrowed_books` and `first_indictors`.
- **Declared foreign key** means a constraint enforced by MySQL. **Implicit reference** (written FK* in the diagrams) means a column that the application treats as a reference but the database does not enforce.
- Diagrams use crow's-foot notation (Chen 1976; Martin 1990): the crow's foot marks the "many" side; solid lines are enforced by the database, dashed lines only by code, dotted lines are identifiers embedded inside JSON.
- Row counts come from the operational database on 15 October 2026. It is a development snapshot (for example, 34 catalogue records and 5 accounts), so counts illustrate structure rather than production volume.
- Findings are numbered F1 to F20 and recommendations R1 to R16 so they can be cited.

2 Methodology

2.1 Research questions

The study was organised around five questions:

1. **RQ1.** Which entities and relationships does the schema realise, and how are they organised into subject areas?
2. **RQ2.** Which semantics are encoded outside the schema, in status codes, JSON documents and application rules?
3. **RQ3.** How do actions of patrons, staff and integrated services manifest as changes to the data?
4. **RQ4.** To what extent does the design satisfy the principles of relational design (entity, referential and domain integrity; normalisation; atomicity), and where does it deviate?
5. **RQ5.** Which changes would most improve integrity, performance and privacy, at what cost?

2.2 Evidence and procedure

Four complementary sources were used, each chosen to answer a different question and to compensate for the blind spots of the others.

Table 1. Evidence sources and what each contributes.

Source	Procedure	Answers
Schema introspection	A database was built from the migrations alone (`elibrarydb_test`, as the test suite does) and read through `information_schema`: tables, columns, indexes, foreign keys, views, triggers, events and routines (Appendix F, Q1).	RQ1, RQ4
Drift check	The same introspection was run against the operational database and compared attribute by attribute: tables, column types, nullability, defaults, keys and indexes.	Validity
Static code analysis	Each of the 61 Eloquent models was resolved to its table by the framework itself (`getTable()`). Every table name and model was traced through controllers, services, helpers and views to classify reads and writes; key flows were then read in full.	RQ2, RQ3
Aggregate data profiling	Row counts and integrity queries (orphaned references, projection consistency, duplicate keys) were run against the operational data. Only counts were collected; no personal record was read (Appendix F, Q2-Q3).	RQ4

The drift check found **no differences** between the migrated and the operational schema (0 columns, 0 indexes and 0 tables differ), so every structural statement in this document applies to both.

2.3 Analytical framework

The description follows the classical three-level separation of conceptual, logical and physical models (ANSI/X3/SPARC 1975; Elmasri and Navathe 2016), with entity-relationship modelling for the conceptual level (Chen 1976). The analysis applies Codd's relational model and normal forms (Codd 1970, 1972; Kent 1983); a four-part integrity taxonomy of entity, referential, domain and transactional integrity (Date 2004); an access-path review that compares the predicates used by the application

with the indexes available; and the data-privacy principles of the Philippine Data Privacy Act of 2012 (Republic Act No. 10173). MARC structures are interpreted against the MARC 21 Format for Bibliographic Data (Library of Congress).

2.4 Threats to validity

- **Volume.** The operational database is a development snapshot. Integrity results (for example, zero orphans) show the current state, not a guarantee; performance conclusions are therefore argued from access paths and growth patterns rather than measured.
- **Static analysis.** Tracing table usage by name can miss dynamically built SQL and can over-count incidental string matches. All findings that matter were confirmed by reading the relevant code in full.
- **External consumers.** The RAG worker lives in a separate repository; its use of the database was established from the producer side (`App\Services\RagSync`) and the project documentation only.
- **Intent of legacy structures.** For empty, unreferenced tables the original purpose is inferred from names and columns, and is labelled as such.

3 Architecture of the data layer

3.1 Platform and context

The database runs on MySQL 8.0.46 and is accessed exclusively by the Laravel 13 application, through Eloquent models and the query builder. Sessions are kept in files, not in the database. Three other systems hold copies of database content: Meilisearch indexes `view_catalogs` through Laravel Scout; a Python RAG worker (repository `hermes-with-rag`) reads `rag_sync_queue` and maintains a LanceDB vector index; and the file system under `public/` stores the PDFs and cover images whose paths are recorded in `multimedia` and `frontpages`. Figure 1 shows these relationships.

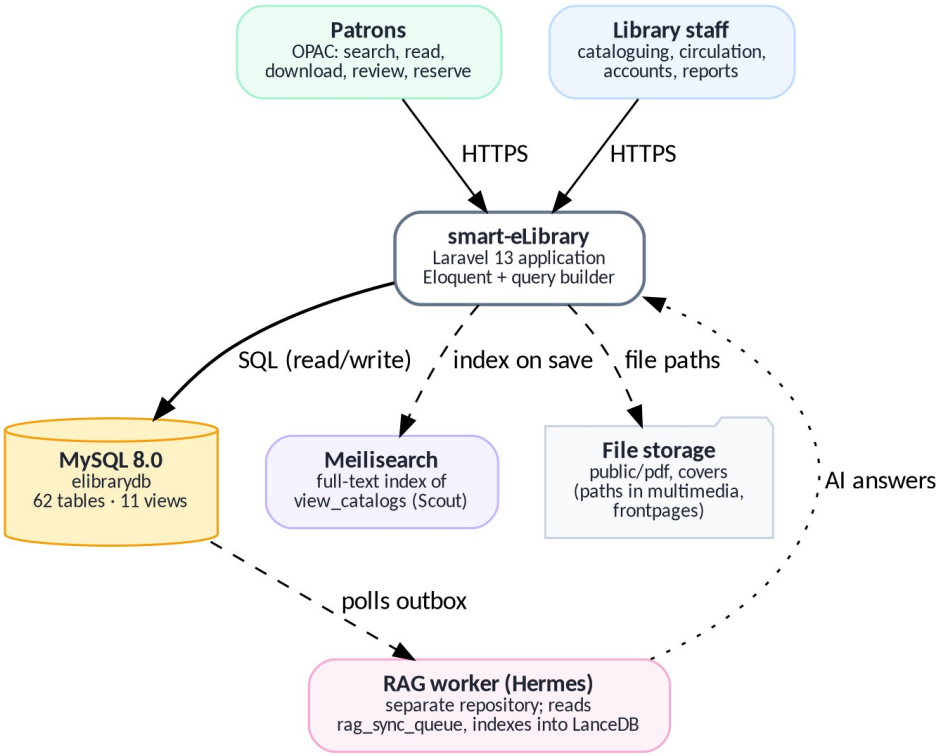


Figure 1. System context: the database, the application that owns it, and the three external stores that hold derived copies of its content.

3.2 Physical characteristics

Table 2 summarises the physical schema. Two figures stand out: the database declares only 6 foreign keys, and it contains no triggers, stored routines or scheduled events, so all behaviour beyond storage lives in the application.

Table 2. Physical summary of the schema.

Measure	Value
Database objects	73: 62 base tables and 11 SQL views
Columns	763 in total, 582 in base tables
Primary keys	59 of 62 base tables (`password_resets`, `view_total_copies`, `visits` have none)
Secondary indexes	24

Measure	Value
Declared foreign keys	6 (four on the Spatie permission pivots, two on `saved_records`)
Triggers, routines, events	0, 0, 0
Objects named view_*	13: 11 SQL views and 2 base tables (`view_catalogs`, `view_total_copies`)

Table 3. Storage engines and collations of the base tables.

Engine / collation	Tables
InnoDB / utf8mb4_unicode_ci	41
InnoDB / utf8mb4_general_ci	15
InnoDB / latin1_swedish_ci	5
MyISAM / utf8mb3_general_ci	1

3.3 Provenance and evolution

The schema has a dual origin. Migration `2025_09_15_114348_drop_all_tables` drops every table, and `2025_09_15_114349_migration` then executes a SQL dump (`database/sql/elib_new.sql`) that creates 56 tables and 11 views: the legacy system's structure carried forward wholesale. Fifteen later migrations, dated July to October 2026, together with Laravel's own 2019 token migration, add six tables (`inquiry_seens`, `rag_sync_queue`, `material_type_maps`, `saved_records`, plus Laravel's `migrations` and `personal_access_tokens`) and alter others (Appendix D).

The dump has itself been refreshed over time: it already contains three tables that later migrations also create (`reads`, `reservation_messages`, `settings`). Those migrations guard with `Schema::hasTable`, so both paths converge, but the schema now has two sources of truth that must be kept in step, and the baseline begins with an unconditional DROP of every table.

4 Conceptual and logical model

4.1 Subject areas

The 73 objects fall into eight subject areas (Figure 2). Two entities act as hubs: the bibliographic record (`catalogs`), referenced by at least 12 columns in other tables, and the account (`users`), referenced by at least 10. Of the 73 objects, 18 belong to superseded designs or are unused; they are shown in grey and discussed in section 4.8.



Figure 2. The eight subject areas and their tables. Bold: core tables; italic: SQL views; grey: legacy or unused.

4.2 Bibliographic description and cataloguing

The central entity is the bibliographic record. Each row of `catalogs` holds one title, and its descriptive content is a complete MARC 21 record serialised as JSON in `field_data` (section 5.1). The relational columns carry only administration: material type, classification system, the import batch that created the record, whether it is published on the OPAC, the AANR-sector flag and a soft-delete trail.

Because MARC JSON cannot be searched, sorted or displayed efficiently, every record has a one-to-one projection in `view_catalogs`. Despite its name this is a base table, written by the cataloguing code in the same request as the record: it flattens title, authors, subjects, publisher, year and call number into columns, and it is the model Laravel Scout indexes into Meilisearch. In architectural terms the pair forms a *write model* and a *read model*, a pattern known as command-query responsibility segregation (Fowler 2011). The MARC reference tables (`marctags_loc` with 249 tags, `marctag_subfields_loc` with 2,606 subfields, `marctag_indicators_loc` with 312 indicator values) define the vocabulary the editor offers; the JSON records point into them by id (Figure 3).

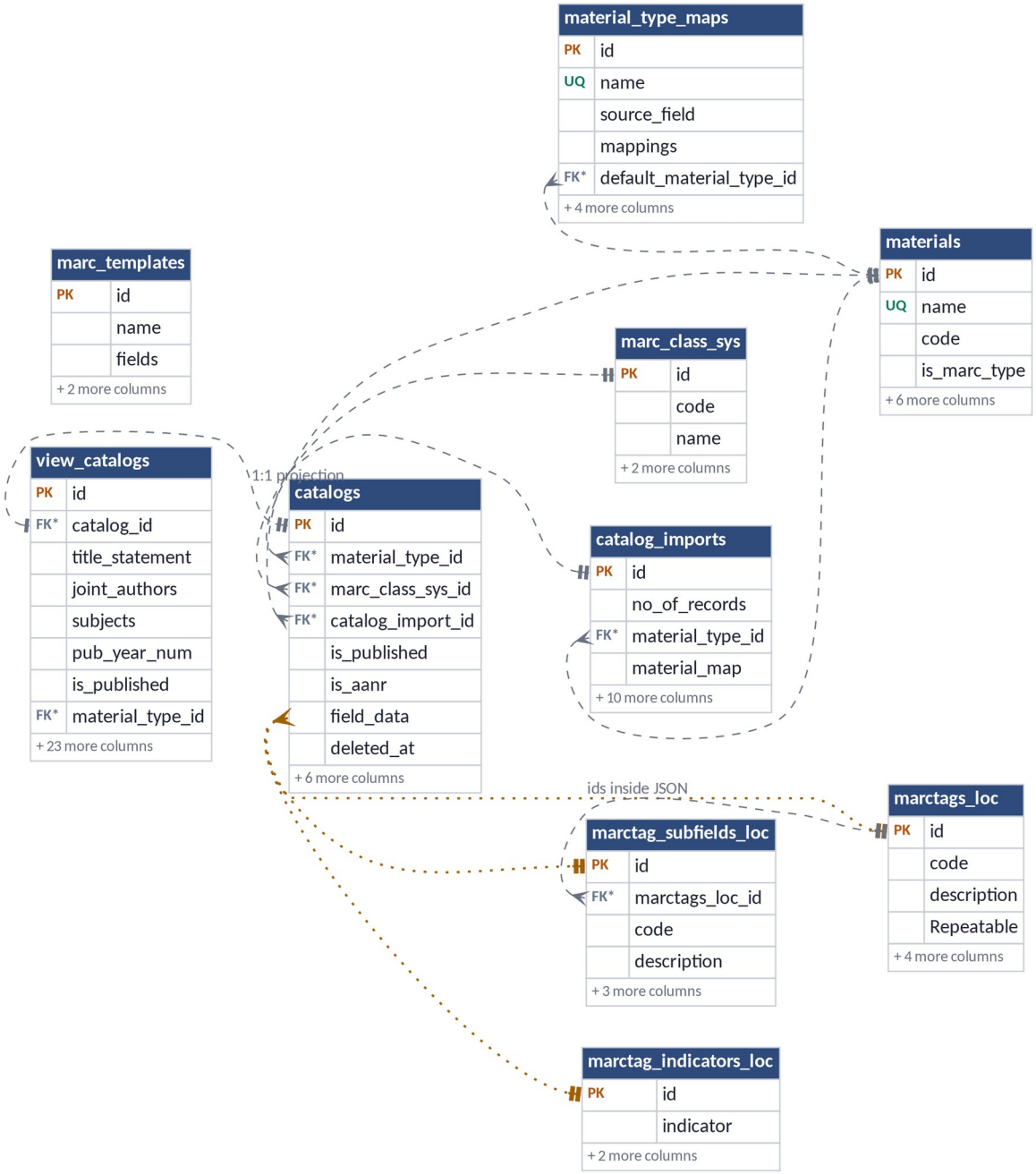


Figure 3. Logical model of bibliographic description and cataloguing.

4.3 Holdings and circulation

A title can have many physical `copies`, each identified in practice by its `circulation_number` (barcode) rather than its key. Circulation is modelled as one transaction row in `circulations` per reservation or loan, keyed for the patron by a `reservation_code` and linked to its fine assessment by a `transaction_code`. Three satellite tables share those codes: `circulation_reservations` (the request details), `circulation_fine_history` (one fine assessment per transaction) and `reservation_messages` (the staff-patron chat). Status vocabularies live in `circulation_status` and `copies_status`, but rows store the code's `order_no`, not its key (section 5.3). Digital objects, the full text (`multimedia`) and the cover (`frontpages`), hang off the title, and `holdingscopypresenteds` logs copies confirmed during stock-taking (Figure 4).

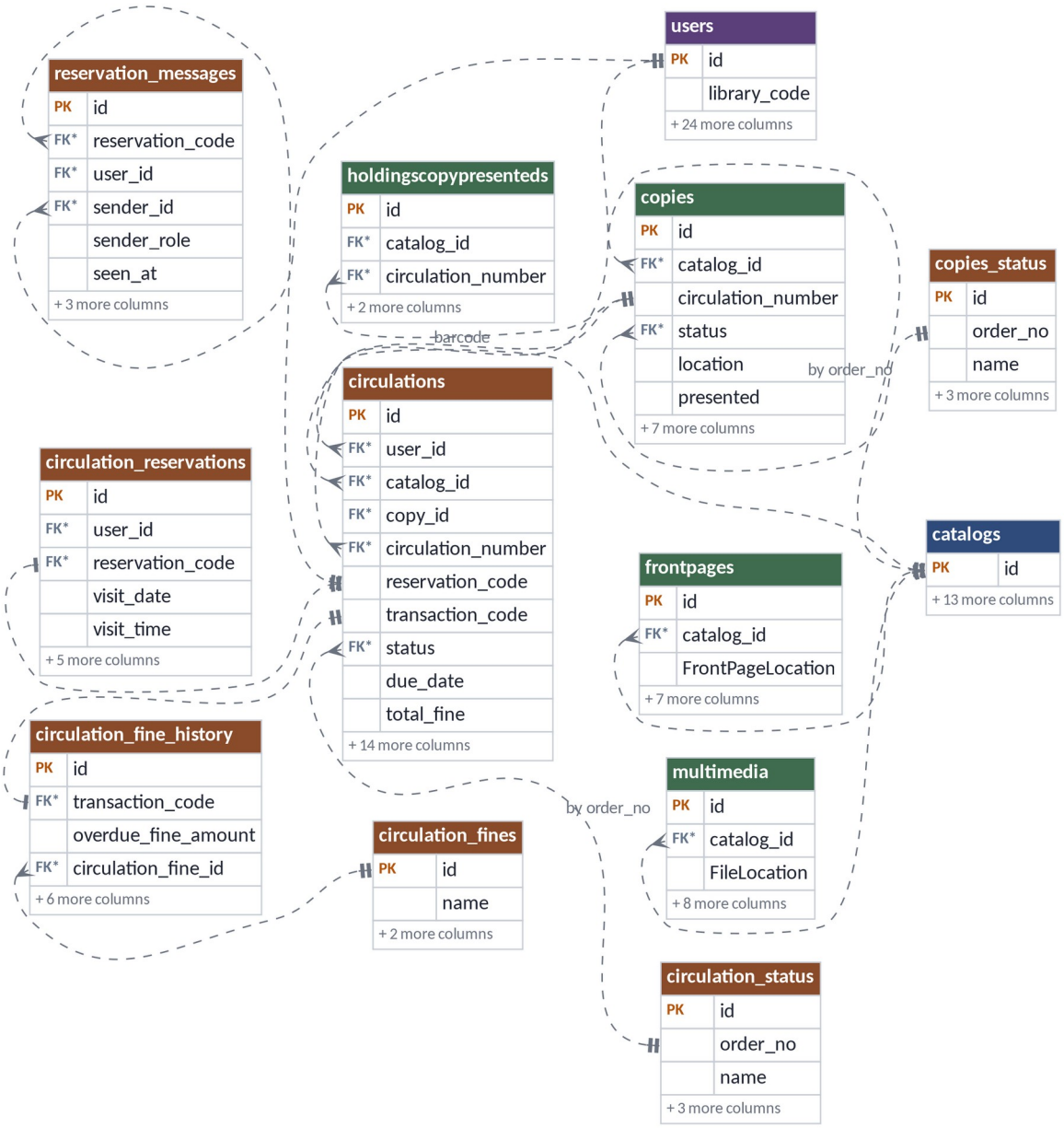


Figure 4. Logical model of holdings, digital objects and circulation. All relationships are implicit; several use natural keys (barcode, reservation and transaction codes).

4.4 People, organisations and access control

One table, `users`, holds every account. `usertype` separates patrons from staff, and staff capabilities come from role-based access control implemented with the Spatie laravel-permission package: `roles` and `permissions` are linked through `role_has_permissions`, and accounts receive roles through the polymorphic pivot `model_has_roles` (Ferraiolo and Kuhn 1992). These pivots carry four of the database's six declared foreign keys. Patron profiling uses `patrontypes` and `regions` (Figure 5).

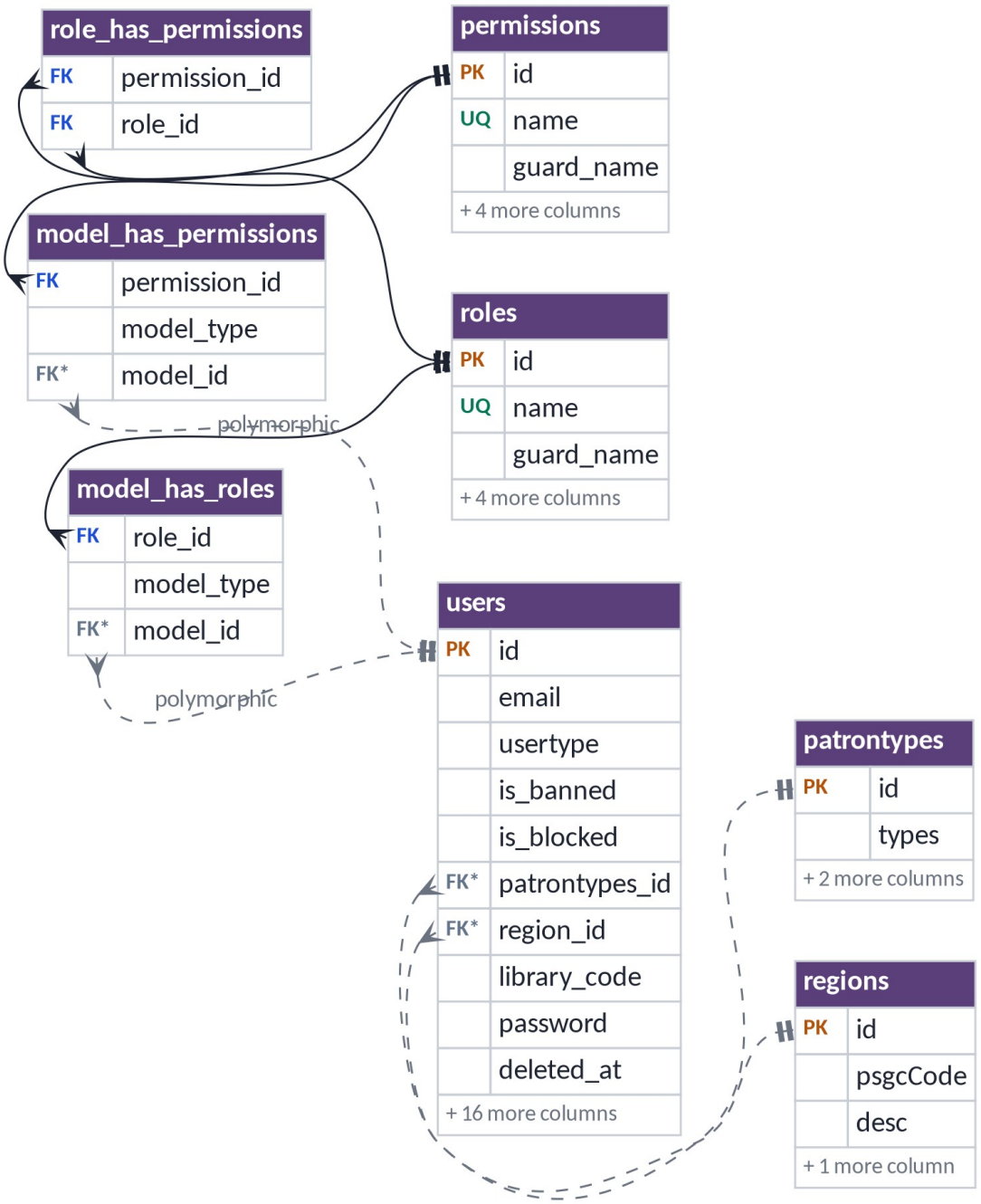


Figure 5. Logical model of accounts and role-based access control. The four solid edges are the only declared foreign keys outside saved_records.

4.5 Engagement and discovery

Four append-only logs record what patrons do with materials: `views` (opening a record page, including anonymous visitors), `reads` (the first online read per patron and material), `downloads` and `ratings` (reviews). `saved_records` holds each patron's saved list. Together they feed view and download counts, the staff dashboard, the To Review list and recommendations of popular titles (Figure 6).

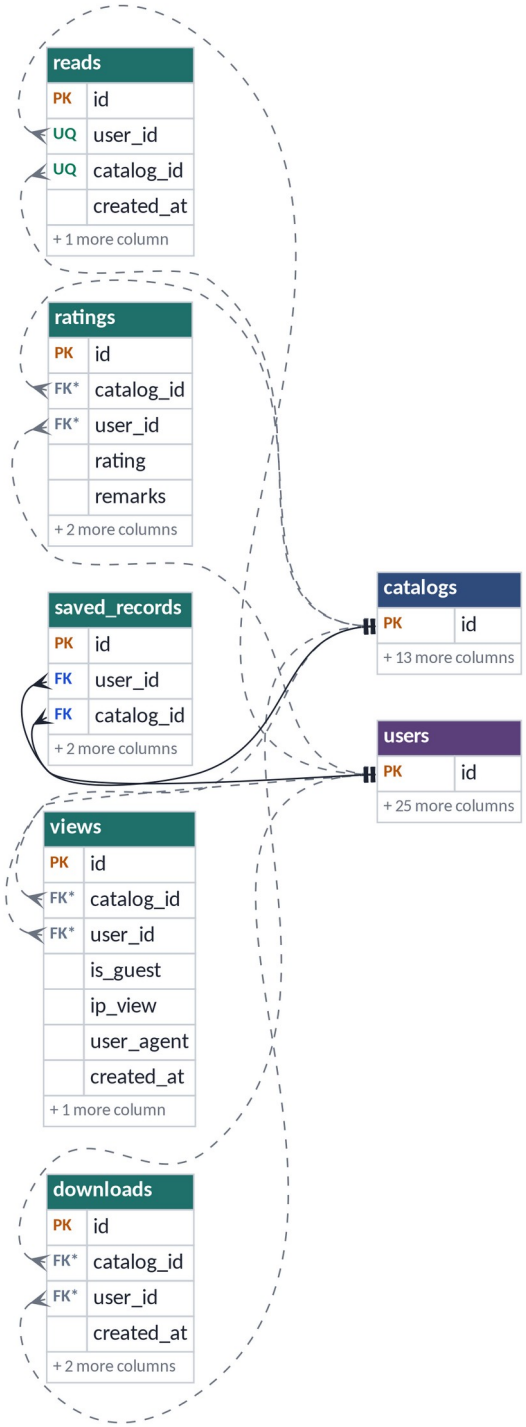


Figure 6. Logical model of engagement. reads and saved_records enforce one row per (user, catalog); the other logs do not.

4.6 Communication and feedback

Registered patrons may send `inquiries`, which become threads of `reply_inquiries`; `inquiry_seens` stores a read receipt per staff member and inquiry and drives the unread badges. `news` holds OPAC announcements, and `csfs` the government Client Satisfaction Feedback form with its seven fixed dimensions (Figure 7).

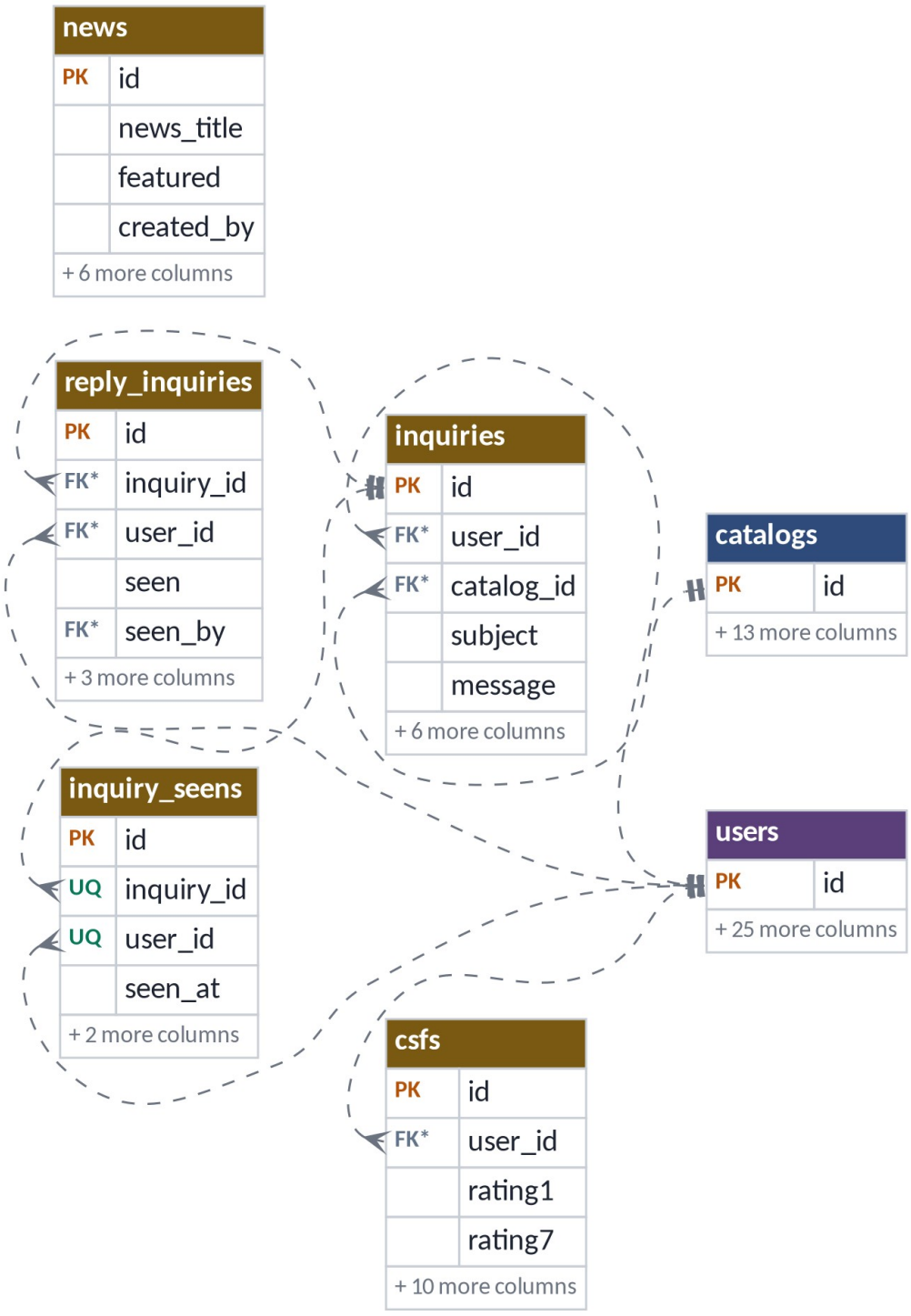


Figure 7. Logical model of inquiries, announcements and satisfaction feedback.

4.7 Integration and configuration

`rag_sync_queue` implements the *transactional outbox* pattern (Richardson 2018): whenever a record changes, the application inserts a row naming the record and the action (`upsert` or `delete`); a separate worker reads pending rows, updates its index and stamps `processed_at`. The producer deduplicates to one pending row per record and never lets a failure abort a catalogue save. `settings` is a key-value store for runtime options. Laravel's own tables (`migrations`, `jobs`, `failed_jobs`,

`password_resets`, `personal_access_tokens`) complete the area; the queue tables are unused because jobs run synchronously.

4.8 Legacy structures

The baseline dump carried forward the previous system's flat bibliographic model, in which a title was one 89-column row in `holdings` with satellite `authors` and `corporate_authors`, and loans were rows in `barrowed_books`. These tables are empty and the application no longer writes them, yet seven SQL views still read them (Figure 8 and finding F13). Several other tables (`thumbnails`, `fullpages`, `p_d_f_s`, `for_reviews`, `view_total_copies`, `visits`, `marc_templates_backup`) are unused.

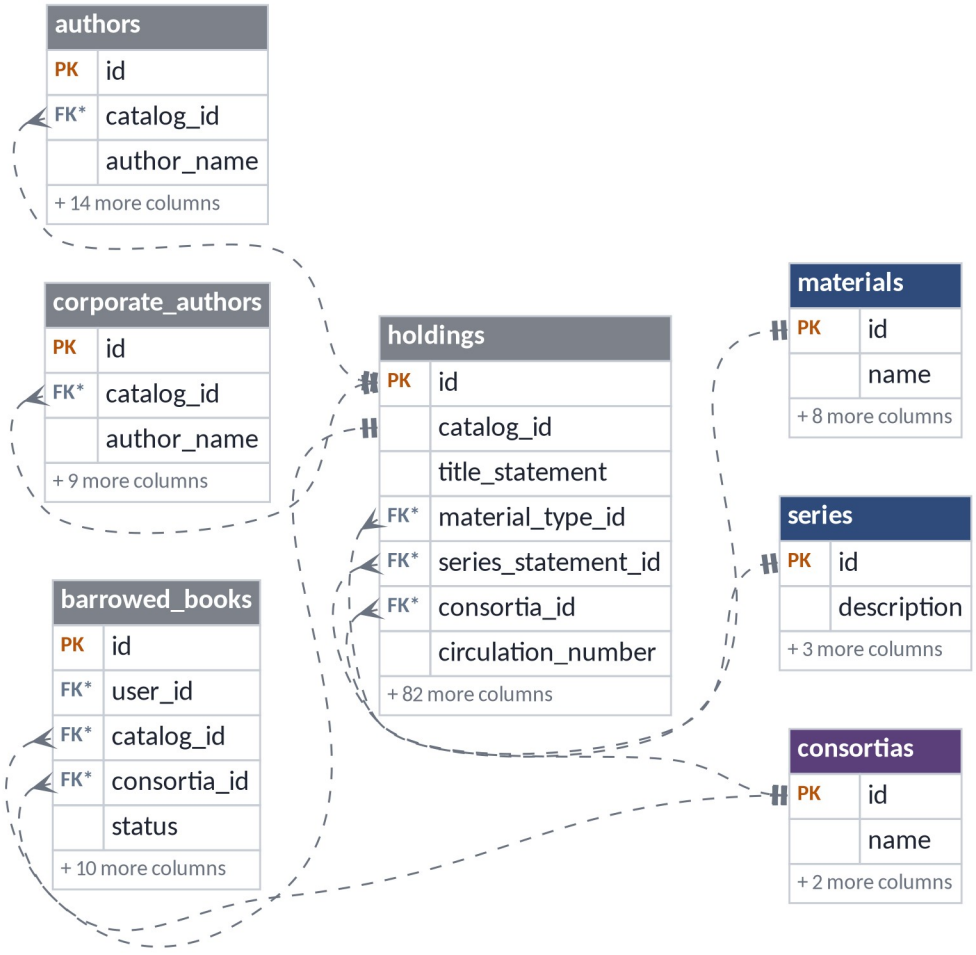


Figure 8. The legacy pre-MARC model, still referenced by seven SQL views.

5 Data semantics: what the schema does not say

5.1 The MARC 21 record in catalogs.field_data

`field_data` is declared as LONGTEXT, not JSON, so MySQL neither validates nor indexes it. Its content is an ordered array with one element per MARC field. Each element has three parts: `marctag` (the tag's id in `marctags_loc`, its code and name), `indicator` (for data fields, the two indicator values with their ids in `marctag_indicators_loc`) and `subfield` (each subfield's id in `marctag_subfields_loc`, its code such as `245$a`, its name and its value). Control fields such as the leader are represented as positional pseudo-subfields (`LDR$06`, Type of record). Figure 9 shows the structure.

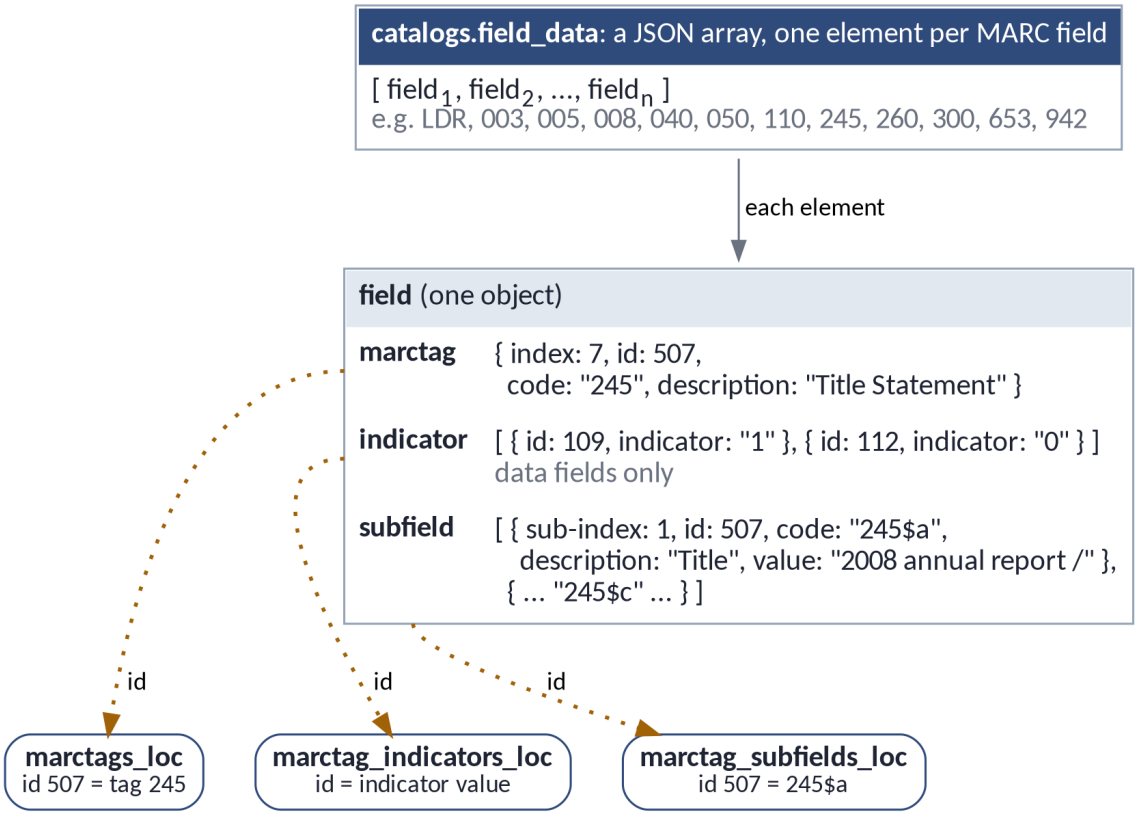


Figure 9. Structure of a MARC record in catalogs.field_data, with the reference tables its ids point to.

The representation is self-describing (names are copied into every record) and preserves field order, repetition and indicators, which a flat relational design could not. The cost is that the references into the MARC vocabulary are invisible to the database, tag and subfield names are copied into every record (records average 6,840 bytes), and any query over bibliographic content must go through the projection.

5.2 The search projection

`view_catalogs` derives its columns from the MARC record: `title_statement` from 245, `joint_authors` from 1XX/7XX joined with " / ", `subjects` from 6XX as a JSON array, `publisher` from 260/264, `pub_year` as transcribed and `pub_year_num` as a number for sorting and range filters, and so on (Appendix A). It is maintained by application code in the same request as the record, and

four Artisan commands (`catalog:rebuild-search-fields`, `catalog:rebuild-titles`, `catalog:rebuild-authors`, `catalog:sync-material-types`) rebuild it from `field_data`. Profiling found every catalogue record projected exactly once, with publication and deletion state in agreement.

5.3 Status vocabularies and flags

Several columns hold codes whose meaning is defined elsewhere. Table 4 gathers them.

Table 4. Coded attributes and their vocabularies.

Column	Values and meaning	Defined in
`circulations.status` (INT)	1 For Approval; 2 Approved; 3 Open (checked out); 4 Declined; 5 Closed (returned, fines settled)	`circulation_status.order_no`
`copies.status` (CHAR(50))	1 Available; 2 Reserved; 3 Borrowed; 4 Not Available; 5 Deleted; 6 Lost; 7 For Repair. Default '4'.	`copies_status.order_no`
`catalogs.is_published`	1 visible on the OPAC; 0 on hold (staff only)	Application
`users.usertype` (ENUM)	Patron, Administrator, Developer; Student is declared but unused	Schema enum
`users.is_banned` / `is_blocked`	Banned: cannot log in or read. Blocked: cannot use chat or inquiries	Application
`views.is_guest` (ENUM)	'YES' anonymous visitor; 'NO' signed in	Schema enum
`rag_sync_queue.action`	'upsert' sync to current state; 'delete' ensure absent	Application
`reservation_messages.sender_role`	'admin' or 'patron'	Application

5.4 The circulation lifecycle as a state machine

Reading the circulation controllers reveals a finite-state machine over `circulations.status` coupled to the status of the copy (Figure 10). A patron's reservation enters state 1. Staff approval moves it to 2 and the copy to Reserved, but only if the copy is Available at that moment. Check-out moves it to 3 and the copy to Borrowed, and sets the due date and an hourly overdue rate. Check-in moves it to 5, returns the copy to Available and records fines (overdue fine = rate × hours overdue, plus any damage fine). Staff can also check out a walk-in loan directly into state 3. Two transitions lead to Declined: a staff decision, and banning the patron, which declines all of their reservations in states 1 and 2.

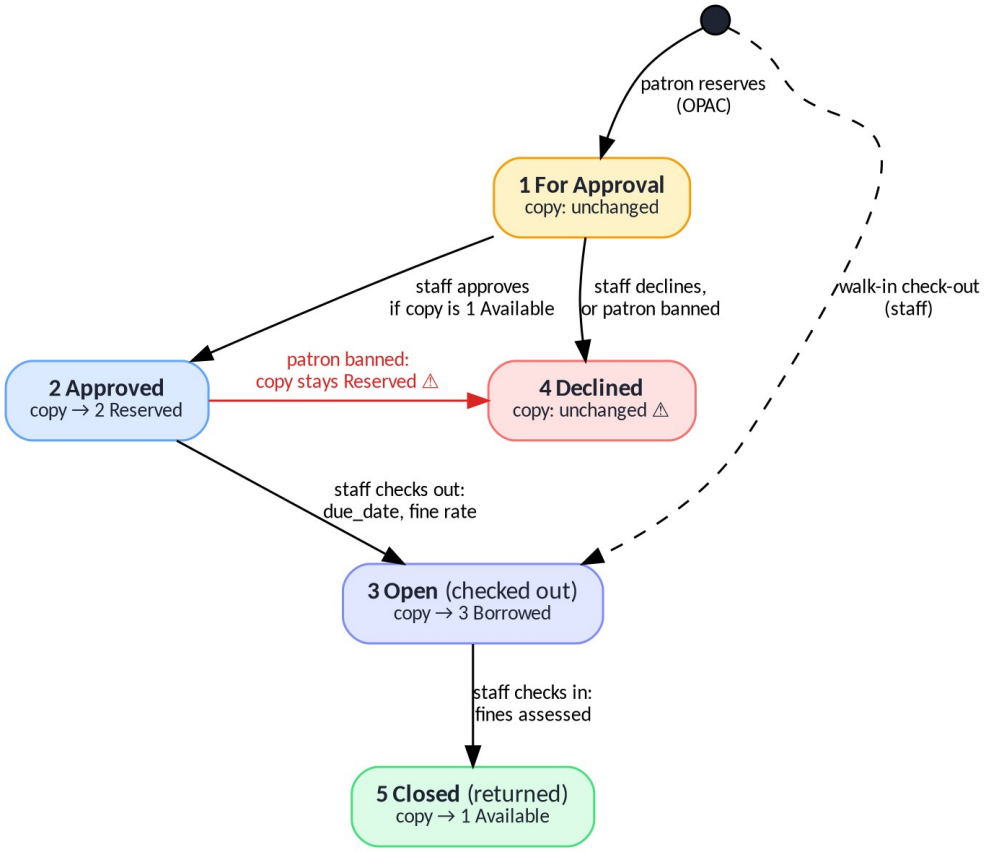


Figure 10. Circulation lifecycle. Each transition also changes the copy's status; the red transition leaves the copy Reserved (finding F11).

5.5 Access-control model

Authorisation is role-based: the application checks 35 permissions (for example `access.dashboard`, `create.catalog`, `circulation.reservation.view`), assigned to 7 roles through 124 grants; super_admin and developer hold all 35, system_admin 29, cataloging_admin 10, circulation_admin 7, faculty 6 and serial_admin 2. Routes check permissions with the `can:` middleware, and a `Gate::before` rule lets the developer role pass every check, including permissions added later. Direct grants to individual users (`model_has_permissions`) are supported but unused.

6 Event model: how actions become data

This chapter answers RQ3. Table 5 lists every application event that writes to the database, the tables it touches and whether the writes are atomic. "No" in the last column means the event issues several statements outside a database transaction, so a failure between them leaves partial state (finding F9).

Table 5. Event catalogue: every write path into the database.

#	Event	Actor	Rows written	Atomic
E1	Register an account	Visitor	`users` +1 (type from default Patron; library card code generated)	Yes
E2	Log in / out	User	None (file sessions). A banned account is signed out at once.	-
E3	Open a record page	Anyone	`views` +1 (guest flag, IP, user agent)	Yes
E4	Read online	Patron	`reads` +1 on the first read of that material	Yes
E5	Download full text	Patron	`downloads` +1	Yes
E6	Review (form or quick popup)	Patron	`ratings` +1 (one per patron per material, checked in code)	Yes
E7	Save / unsave	Patron	`saved_records` +1 or -1	Yes
E8	Inquire and reply	Patron, staff	`inquiries` +1; `reply_inquiries` +1 per message; `inquiry_seens` upsert when staff open a thread	Per statement
E9	Reserve a material	Patron	`circulations` +1 (state 1); `circulation_reservations` +1; `circulation_fine_history` +1 (empty)	No
E10	Approve a reservation	Staff	`circulations` state 1→2 with copy barcode and approver; `copies` 1→2	No
E11	Decline a reservation	Staff	`circulations` → 4 with decliner	Yes
E12	Reservation chat	Staff, patron	`reservation_messages` +1; `seen_at` set on reading	Yes
E13	Check out	Staff	`copies` → 3; `circulations` → 3 (or +1 for a walk-in) with due date and rate; `circulation_fine_history` +1 for walk-ins	No
E14	Check in	Staff	`copies` → 1; `circulations` → 5 with receiver and fines; `circulation_fine_history` updated	No
E15	Ban or block a patron	Staff	`users` flag; on ban, `circulations` in states 1-2 → 4	No
E16	Create or edit a record	Staff	`catalogs` and `view_catalogs` written; Meilisearch updated; `rag_sync_queue` +1 (deduplicated)	No
E17	Publish, hold or delete	Staff	`catalogs` and `view_catalogs` flags or `deleted_at`; removed from or added to the index; `rag_sync_queue` +1	No
E18	Batch import MARC	Staff	`catalog_imports` +1; `catalog_import_progress` updated; per record `catalogs` and `view_catalogs` +1; `rag_sync_queue` +n	No
E19	Attach full text or cover	Staff	`multimedia` or `frontpages` +1/update/soft delete, plus the file on disk	No
E20	Inventory count	Staff	`holdingscopypresenteds` +1; `copies.presented` stamped (Available copies only)	No
E21	Manage roles, settings, news	Staff	`roles`, `permissions`, pivots; `settings` upsert; `news`	Per statement
E22	RAG synchronisation	RAG worker	`rag_sync_queue.processed_at`, `attempts`, `error`	External

#	Event	Actor	Rows written	Atomic
E23	Satisfaction feedback	Patron	`csfs` +1	Yes

6.1 Patron engagement

Figure 11 follows a patron from discovery to review. Each step appends one row to its own log, and the To Review list is not stored at all: it is computed on demand as the set of materials the patron downloaded or read, minus those already rated. A review therefore removes a material from the list without any update.

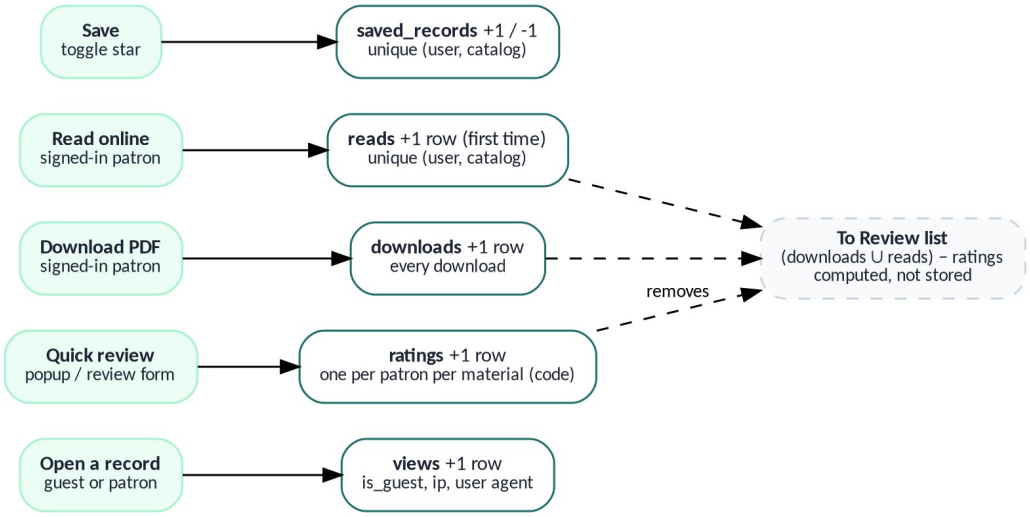


Figure 11. Patron engagement events and the rows they write. The To Review list is derived, not stored.

6.2 Catalogue propagation

A catalogue change fans out to four stores (Figure 12). The record and its projection are written to MySQL in the same request, Scout updates Meilisearch, and a row in the outbox schedules the RAG worker. Only the database writes are under the application's control; the other two are eventually consistent. The outbox makes the RAG side recoverable after a crash, but the database writes are not wrapped in a transaction, so a failure between them would leave the record and its projection out of step until a rebuild command is run.

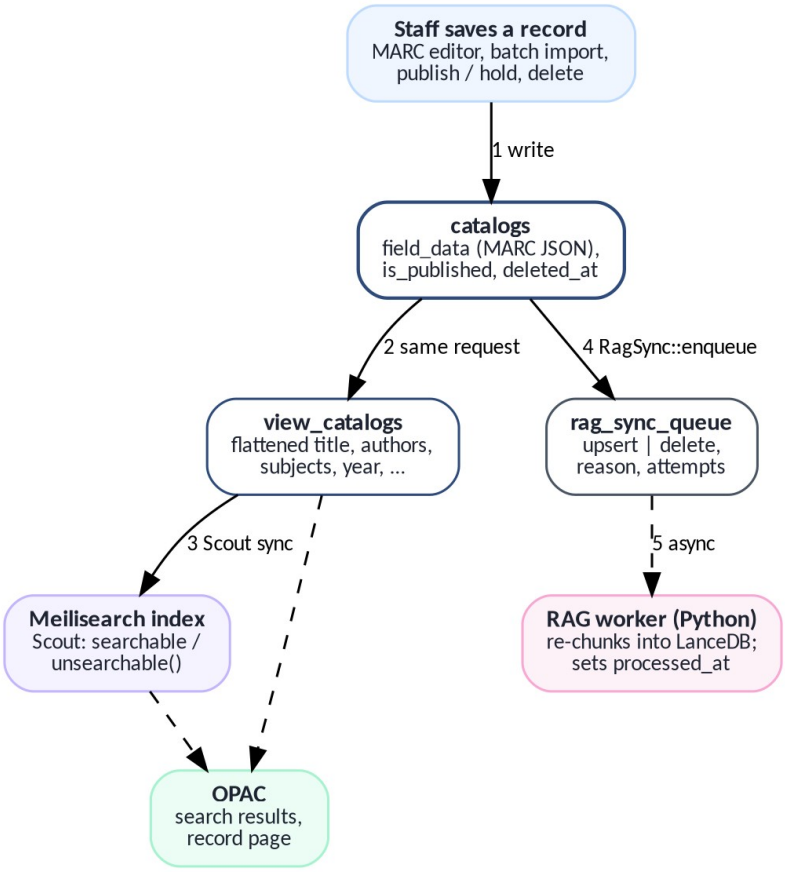


Figure 12. How a catalogue change propagates: one write fans out to the projection, the search index and the RAG outbox.

7 Analysis

This chapter answers RQ4. Each finding states what was observed, the evidence, and why it matters. Table 6 summarises them by severity: **high** findings can corrupt data or break features now, **medium** ones weaken integrity or performance as the data grows, and **low** ones affect maintainability.

Table 6. Summary of findings.

#	Finding	Severity
F1	Referential integrity is almost entirely implicit	High
F2	One identifier, seven physical types	High
F3	Money and dates stored as text	Medium
F4	Codes without constraints; sentinel value 0	Medium
F5	TINYINT import-batch reference caps at 127	Medium
F6	Tables without primary keys; redundant indexes	Low
F7	Natural keys not unique (email, barcode, codes)	High
F8	Hot predicates without indexes	Medium
F9	Multi-step operations are not atomic	High
F10	Check-then-act race on approval	Medium
F11	Declining an approved reservation leaks the copy	High
F12	Staff recorded by name, not identity	Low
F13	Schema-code drift: missing views, views on empty tables	High
F14	NULL comparison defect in a view	Low
F15	Mixed character sets, collations and engines	Medium
F16	Legacy and unused structures	Low
F17	MARC stored as unvalidated LONGTEXT	Medium
F18	Projection maintained only by application code	Medium
F19	Personal data retained without a policy	Medium
F20	Repeating groups and redundant copies	Low

7.1 Entity and referential integrity

F1. Referential integrity is almost entirely implicit

Of the relationships the application relies on, only 6 are declared: four on the permission pivots and two on `saved_records`. The profile in Appendix F (Q2) checked 30 references, of which 28 are implicit and not enforced by MySQL. The current data are clean, with no orphaned rows except two accounts whose `patrontypes_id` is 0 (F4), but nothing prevents a deleted record or account from leaving dangling rows, and the soft-delete convention (rows marked `deleted_at` rather than removed) hides the problem rather than solving it. Without declared keys, the database cannot protect relationships against bugs, manual edits or future code paths, and tools cannot discover the model.

F6. Tables without primary keys; redundant indexes

`password_resets`, `view_total_copies` and `visits` have no primary key, which violates entity integrity and makes individual rows impossible to address reliably. Conversely, `circulations`, `circulation_fines` and `circulation_fine_history` carry a UNIQUE index on `id` in addition to the primary key on the same column: redundant indexes that cost space and write time and add nothing.

F7. Natural keys are not unique

Several attributes act as identifiers in the application but are not declared unique: `users.email` (the login identifier, unique only by form validation, so two concurrent sign-ups can create duplicates), `copies.circulation_number` (the barcode that circulation joins on), `circulations.reservation_code` and `transaction_code` (the keys of three satellite tables) and `view_catalogs.catalog_id` (a one-to-one link). `ratings` relies on code for its one-review-per-patron rule, whereas the comparable `reads` and `saved_records` enforce theirs with unique indexes.

7.2 Domain integrity

F2. One identifier, seven physical types

The identifier of a catalogue record is stored in 19 columns with 7 different types (Table 7), and user references show the same pattern (`users.id` is BIGINT UNSIGNED; `views.user_id`, `downloads.user_id`, `ratings.user_id` and `circulations.user_id` are INT; `catalogs.created_by` stores user ids as text). Comparing a string column with an integer makes MySQL convert each row's value before comparing, which prevents the use of an index on the string side; equal identifiers can compare unequal as strings ('1147' and '01147'); and a foreign key cannot be declared between columns of different types. This is the single largest obstacle to remedying F1.

Table 7. Physical types of columns that hold a catalogue record id.

Column	Type
`catalogs.id` (the key)	int unsigned
`authors.catalog_id`	varchar(24)
`barrowed_books.catalog_id`	varchar(50)
`circulations.catalog_id`	int
`copies.catalog_id`	varchar(50)
`corporate_authors.catalog_id`	varchar(50)
`downloads.catalog_id`	varchar(50)
`frontpages.catalog_id`	varchar(24)
`holdings.catalog_id`	varchar(24)
`holdingscopypresenteds.catalog_id`	varchar(100)
`inquiries.catalog_id`	varchar(50)
`multimedia.catalog_id`	varchar(24)
`p_d_f_s.catalog_id`	int
`rag_sync_queue.catalog_id`	bigint unsigned
`ratings.catalog_id`	varchar(25)

Column	Type
reads.catalog_id	bigint unsigned
saved_records.catalog_id	int unsigned
view_catalogs.catalog_id	int unsigned
view_total_copies.holdings_id	varchar(24)
views.catalog_id	varchar(50)

F3. Money and dates stored as text

Every monetary amount in circulation is VARCHAR(50): `overdue_fine_rate`, `waived_amount`, `total_fine` and `total_fine_paid` in `circulations`, and the fine amounts in `circulation_fine_history`. So are several dates: `users.birthdate`, `circulation_reservations.visit_date` and `visit_time`, `view_catalogs.catalog_date`. Text admits values that are not numbers or dates, sorts lexically ('100' before '20'), and pushes validation and arithmetic into application code.

F4. Codes without constraints; sentinel value 0

`circulations.status` and `copies.status` are bare codes with no foreign key or CHECK constraint to their vocabularies, and the two are typed differently (INT versus CHAR(50) holding digits). `users.patrontypes_id` is NOT NULL and staff accounts use 0 to mean "none", a value no patron type has; this sentinel is why two rows appear as orphans in the profile and why a foreign key cannot be added as is.

F5. A TINYINT reference caps import batches at 127

`catalogs.catalog_import_id` and `view_catalogs.catalog_import_id` are signed TINYINT columns referencing `catalog_imports.id`, an INT UNSIGNED key. The 128th batch import will fail to store its batch id (an out-of-range error in strict mode, a silently clipped value otherwise), and every record it creates will point at the wrong batch.

7.3 Transactional integrity and concurrency

F9. Multi-step operations are not atomic

The events marked "No" in Table 5 each issue several dependent statements without `DB::transaction`. A reservation writes three rows; approval updates the transaction and the copy; check-out and check-in update the copy, the transaction and the fine record; a catalogue save writes the record and its projection. A failure between statements (a validation exception, a timeout, a deadlock) leaves partial state, for example a copy marked Borrowed whose transaction is still Approved. The `regions` table uses MyISAM, which supports no transactions at all, although as read-only reference data it is not at risk.

F10. Check-then-act race on approval

Approval reads the copy, checks that its status is 1 (Available), then updates the transaction and the copy in separate statements without locking. Two staff approving reservations for the same copy at the same moment can both pass the check, leaving two approved reservations for one copy: a lost-update anomaly (Gray and Reuter 1993). A conditional update (`UPDATE copies SET status = 2 WHERE`

`circulation_number = ? AND status = 1`, checking the affected-row count) or `SELECT ... FOR UPDATE` inside a transaction removes the race.

F11. Declining an approved reservation leaks the copy

Approval moves the copy to Reserved, but neither a staff decline nor the automatic decline that follows a ban moves it back. Banning a patron declines all of their reservations in states 1 and 2, so any copy they had approved stays Reserved indefinitely and disappears from availability (red transition in Figure 10). The defect is in the application, but it violates a database-level invariant that the schema could state and enforce: a copy is Reserved only while an approved reservation references it.

7.4 Consistency between schema and code

F13. Missing views, and views over empty legacy tables

Four Eloquent models point at tables or views that do not exist: `View_inventory_missing` and `View_inventory_presented` (used by the Inventory "Missing" and "Presented" report pages, which therefore fail with "table not found"), and `Result` and `User_backup` (unused). In the other direction, seven of the 11 SQL views read the empty legacy tables `holdings` or `barrowed_books`. Only one of them is still read by the application: `view_inquiries`, whose title column comes from `holdings` and is therefore always NULL (the inquiry page works around this by looking the title up separately). `view_to_reviews` has the same defect, and `view_searches`, the table behind the `Search` model, returns no rows at all; neither model is used by the current code.

F14. A NULL comparison defect

`view_borrowed_books` computes the length of a loan with `IF(a.returned_at = NULL, ..., ...)`. Under SQL's three-valued logic a comparison with NULL is never true (Codd 1979), so the expression always takes the second branch and reports the days elapsed until today even for returned loans. The correct test is `a.returned_at IS NULL`. The view reads a legacy table, so the defect has no current effect, but it would if the view were revived.

F18. The projection is maintained only by application code

The consistency of `view_catalogs` with `catalogs` depends on every write path remembering to update both. The profile found them consistent (0 records without a projection, 0 publication or deletion mismatches), and rebuild commands exist as a repair path, but a single missed path, or the absence of a transaction (F9), would let search results diverge from records silently.

7.5 Physical design and performance

F8. Hot predicates without indexes

Comparing the predicates the application uses with the indexes available shows several frequent access paths that must scan whole tables (Table 8). The most important are `view_catalogs.catalog_id`, looked up on every record page, and the append-only logs `views` and `downloads`, filtered by record, user and date for counts and the dashboard. These logs grow without bound, so their cost grows linearly with use. The dashboard's two-minute cache masks the effect today.

Table 8. Frequent predicates without a supporting index.

Table	Predicate used by the application	Used for
`view_catalogs`	`catalog_id = ?`	Every record page, citations, admin record view
`views`	`catalog_id = ?`; `created_at BETWEEN ...`	View counts, most viewed, dashboard
`downloads`	`catalog_id = ?`; `user_id = ?`; `created_at BETWEEN ...`	Counts, To Review, dashboard
`ratings`	`catalog_id = ?`; `(user_id, catalog_id)`	Record page averages, one-review rule
`copies`	`circulation_number = ?`	Approval, check-out, check-in, inventory
`circulations`	`reservation_code = ?`; `transaction_code = ?`; `status IN (...)`	All circulation screens, dashboard
`inquiries`	`user_id = ?`	Patron inquiry list, notifications

F15. Mixed character sets, collations and engines

Base tables use four engine and collation combinations (Table 3). Five tables (`authors`, `frontpages`, `holdings`, `materials`, `multimedia`) use latin1, which cannot store characters outside the Western European repertoire. The utf8mb4 tables are split between two collations, `utf8mb4_general_ci` and `utf8mb4_unicode_ci`, so comparing string columns across them can fail with "Illegal mix of collations" or force a conversion that disables indexes, a risk compounded by F2 because many identifier columns are strings. `regions` alone uses MyISAM with utf8mb3.

F17. MARC stored as unvalidated LONGTEXT

Declaring `field_data` as LONGTEXT rather than MySQL's native JSON type means malformed documents are accepted, and the type offers neither path indexing nor generated columns. Embedding the MARC structure as a document is a defensible choice, since MARC's repeatable, ordered and optional fields fit a relational table poorly and the projection handles querying, but the document deserves the validation the JSON type provides.

7.6 Normalisation

Table 9 assesses the core tables against the normal forms. The bibliographic core is deliberately not in first normal form: `field_data` is a composite document, and `view_catalogs` is a derived read model in which `joint_authors` and `subjects` are multi-valued. Both are justified by their roles. The remaining departures are of two kinds. *Historical snapshots*, such as the requester's name copied into a reservation at the time of the request, are legitimate, because they record a fact as it was. *Redundant copies* that should always equal their source are not, because they create update anomalies: `copies.title`, `circulations.user_library_code` and the contact details copied into `inquiries`.

Table 9. Normal-form assessment of the core tables.

Table	Assessment	Departure and its status
`catalogs`	Below 1NF (by design)	`field_data` is a composite document; relational columns are in 3NF
`view_catalogs`	Derived; not normalised	Read model; multi-valued `joint_authors`, `subjects` (justified)

Table	Assessment	Departure and its status
`copies`	2NF	`title` depends on `catalog_id` (redundant copy)
`circulations`	2NF	`user_library_code` depends on `user_id` (redundant); staff names (F12)
`circulation_reservations`	3NF	`requested_by`, `title_statement` are snapshots (legitimate)
`circulation_fine_history`	3NF	One-to-one with `circulations`; could be merged
`users`	1NF, repeating group	`position1-position4` (F20)
`inquiries`	2NF	Name, contact and email copied from `users` (redundant)
`csfs`	3NF	`rating1-rating7` are a fixed instrument (acceptable)
Logs, pivots, `rag_sync_queue`	BCNF	None

F20. Repeating groups and redundant copies

`users.position1` to `position4` is a repeating group: it caps an account at four positions and makes "find everyone holding position X" a four-way disjunction. `csfs.rating1` to `rating7` look similar but map one-to-one to the seven fixed dimensions of the government form, so they are best read as distinct attributes with poor names.

7.7 Accountability and maintainability

F12. Staff recorded by name, not identity

Circulation records who approved, declined, released and received a transaction by storing the staff member's *name* (`approved_by` and similar, VARCHAR). Names are neither unique nor stable, so the trail cannot be joined reliably to accounts and silently changes meaning when an account is renamed. `catalogs.created_by` does the opposite, storing an id as text.

F16. Legacy and unused structures

18 of the 73 objects are legacy or unused (Appendix A marks each). They enlarge the surface a maintainer must understand, invite accidental reuse, and are the source of F13 and F14. The 89-column `holdings` table alone accounts for 89 of the schema's 763 columns, and its derived view `view_holdings_uncatalog` for another 90.

7.8 Security and privacy

F19. Personal data retained without a policy

Table 10 inventories the personal data the database holds. Most of it is necessary for the service, but two practices deserve attention under the principles of proportionality and retention in the Data Privacy Act of 2012 (RA 10173, s. 11). First, `views` stores the IP address and full user agent of every page view, including anonymous visitors, indefinitely and in clear. Second, contact details are duplicated into `inquiries`, `circulation_reservations` and the legacy `barrowed_books` and `visits`, multiplying the places where a correction or erasure must be applied. Passwords are stored as bcrypt hashes, and the seed data no longer contains real password hashes (removed in October 2026); real names and contact details of three accounts remain in the seed file in the public repository.

Table 10. Inventory of personal data.

Table	Personal data	Purpose
`users`	Name, email, contact number, birthdate, sex, location, region, affiliation, positions, patron category	Account, eligibility, statistics
`views`	IP address, user agent, account id	View counts, abuse detection
`inquiries`, `reply_inquiries`	Name, contact number, email (copied); message content	Patron support
`circulations`, `circulation_reservations`	Account, card code, requester name, visit schedule; staff names	Lending
`reservation_messages`	Message content between staff and patron	Lending
`ratings`, `csfs`	Opinions linked to an account	Feedback
`downloads`, `reads`, `saved_records`	Reading behaviour linked to an account	Service, To Review
Legacy: `barrowed_books`, `visits`	Name, contact number, email	None (unused)

8 Recommendations

Table 11 turns the findings into a programme ordered by priority. Priority 1 items correct defects that can corrupt data or break features now, and each is small. Priority 2 items restore integrity that the database should enforce; they need data-cleaning migrations and should be done in the order given, because type unification (R5) is the prerequisite for declaring foreign keys. Priority 3 items improve hygiene over the longer term. Every schema change should be shipped as a migration and tested against `elibrarydb_test` before production.

Table 11. Prioritised recommendations.

#	Recommendation	Addresses	Effort
Priority 1			
R1	Add `UNIQUE(users.email)` after checking for duplicates.	F7	Low
R2	Repair the inventory reports (create the two views or query `copies` and `holdingscopypresenteds` directly); re-point `view_inquiries` and `view_to_reviews` at `view_catalogs`; drop `view_searches`.	F13	Low
R3	On decline and on ban, return a Reserved copy to Available; approve with a conditional update that checks the affected-row count.	F10, F11	Low
R4	Wrap every multi-statement event (E9, E10, E13-E18) in `DB::transaction`.	F9, F18	Low
Priority 2			
R5	Unify identifier types: every catalogue reference INT UNSIGNED, every user reference BIGINT UNSIGNED.	F2	Medium
R6	Declare foreign keys for the implicit references (RESTRICT for history tables, CASCADE for dependent logs).	F1	Medium
R7	Add the indexes of Table 8, with UNIQUE on `copies.circulation_number`, `circulations.reservation_code` and `transaction_code`, `view_catalogs.catalog_id` and `ratings(user_id, catalog_id)`; drop the redundant UNIQUE(id) indexes.	F6, F7, F8	Low
R8	Store money as DECIMAL(10,2) and dates as DATE or DATETIME; type status codes as TINYINT UNSIGNED with foreign keys or CHECK constraints.	F3, F4	Medium
R9	Widen `catalog_import_id` to INT UNSIGNED in both tables.	F5	Low
R10	Make `users.patrontypes_id` nullable and replace the sentinel 0 with NULL.	F4	Low
R11	Record staff by id (`approved_by_id` and so on) alongside or instead of names.	F12	Medium
Priority 3			
R12	Convert latin1 tables to utf8mb4, standardise one collation (e.g. `utf8mb4_0900_ai_ci`), move `regions` to InnoDB.	F15	Medium
R13	Archive and drop the legacy tables and views after a final backup.	F13, F14, F16	Medium
R14	Change `field_data` to the JSON type; add generated columns only where a MARC path must be filtered directly.	F17	Low

#	Recommendation	Addresses	Effort
R15	Adopt a retention rule for `views` (e.g. truncate or hash IPs after 90 days); remove copied contact details; replace real personal data in seed files with fictitious data.	F19	Low
R16	Give `visits` and `password_resets` primary keys (or adopt Laravel's current `password_reset_tokens`); replace `users.position1-4` with a child table if multiple positions matter.	F6, F20	Low

9 Conclusion

The smart-eLibrary database is the product of two eras: a flat legacy model imported wholesale, and a modern layer built around MARC records, role-based access and integration with search and AI services. The modern layer is coherent. Its central idea, a MARC document as the write model and a flattened projection as the read model, suits bibliographic data well, and the outbox gives the RAG integration a sound recovery story. Current data are clean.

Its weakness is that integrity is a property of the code rather than of the database. Relationships, types, uniqueness, state transitions and atomicity are all enforced, where they are enforced at all, by application logic that must be right on every path. The defects found here, a broken report, a copy that can be lost to availability, a cap of 127 imports, are typical of that arrangement. The recommendations move those guarantees into the schema in a deliberate order: repair what is broken, make identifiers uniform, declare the constraints, then retire the legacy. Each step can be taken independently and verified against a test database, and together they would turn a database that is correct by convention into one that is correct by construction.

References

ANSI/X3/SPARC Study Group on Data Base Management Systems (1975). Interim report. *FDT, Bulletin of ACM SIGMOD*, 7(2).

Chen, P. P.-S. (1976). The entity-relationship model: toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9-36.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.

Codd, E. F. (1972). Further normalization of the data base relational model. In R. Rustin (Ed.), *Data Base Systems* (pp. 33-64). Prentice-Hall.

Codd, E. F. (1979). Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4), 397-434.

Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.

Elmasri, R., and Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.

Ferraiolo, D., and Kuhn, R. (1992). Role-based access controls. In *Proceedings of the 15th National Computer Security Conference* (pp. 554-563).

Fowler, M. (2011). *CQRS*. martinfowler.com/bliki/CQRS.html.

Gray, J., and Reuter, A. (1993). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120-125.

Library of Congress, Network Development and MARC Standards Office. *MARC 21 Format for Bibliographic Data*. loc.gov/marc/bibliographic/.

Martin, J. (1990). *Information Engineering, Book II: Planning and Analysis*. Prentice-Hall.

Oracle Corporation. *MySQL 8.0 Reference Manual*. dev.mysql.com/doc/refman/8.0/en/.

Republic of the Philippines (2012). *Republic Act No. 10173: Data Privacy Act of 2012*.

Richardson, C. (2018). *Microservices Patterns*. Manning.

Appendix A Data dictionary

Every object is listed below by subject area. For each: its purpose, its status (core, derived, reference, framework, legacy or unused), storage and keys, and its columns. "Key" shows PK (primary key), UQ (unique), IX (indexed) and FK (declared foreign key); implicit references are described in the Description column. Rows are from the 15 October 2026 snapshot. Legacy and unused objects with many columns are listed compactly. A default of now() stands for CURRENT_TIMESTAMP.

A.1 Bibliographic description and cataloguing

catalog_import_progress

Progress counter of a running batch import, polled by the import screen.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int unsigned	N		PK	Surrogate primary key.
progress	int	Y	0		Records processed so far.
total_records	int	Y	0		Records in the batch.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

catalog_imports

One row per batch import of MARC records (e.g. a Koha export): record count, subject, default material type and the material-type mapping used.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 1 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int unsigned	N		PK	Surrogate primary key.
no_of_records	int	Y			Records in the batch.
subject	varchar(255)	Y			Batch label.
status	varchar(50)	Y	imported		Batch state ('imported', ...).
is_aanr	tinyint	Y	0		Default AANR flag for the batch.
material_type_id	int	Y	21		Default material type.
material_map	json	Y			JSON: the material-type mapping applied.
comment	text	Y			Free-text note.
created_at	timestamp	Y			Row creation time.
created_by	varchar(50)	Y			Staff account that created the row.
updated_at	timestamp	Y			Last update time.
updated_by	varchar(50)	Y			Staff account that updated the row.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.
deleted_by	varchar(50)	Y			Staff account that deleted the row.

catalogs

The bibliographic record: one row per catalogued title. The full MARC 21 record is stored as JSON in field_data; the remaining columns are administrative (material type, publication state, import batch, audit trail). Source of truth for everything the OPAC and the search index show.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 34 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int unsigned	N		PK	Surrogate primary key.
material_type_id	int unsigned	Y	1		Material type (materials.id; implicit).
is_aanr	tinyint	N	0		1 if the material belongs to the AANR sector (agriculture, aquatic and natural resources).
catalog_import_id	tinyint	Y			Import batch that created the record (catalog_imports.id; implicit). TINYINT: at most 127 batches can be referenced.
is_published	tinyint	N	0		1 = visible on the OPAC; 0 = on hold (staff only).
marc_class_sys_id	int	N	1		Classification system (marc_class_sys.id; 1 LOC, 2 DDC).
accession_number	varchar(255)	Y			Library accession number(s) of the title.
field_data	longtext	Y			The full MARC 21 record as JSON: an ordered array of fields, each with tag, indicators and subfields (see section 5.1).
created_at	timestamp	Y			Row creation time.
created_by	varchar(100)	Y			Staff user id, stored as text.
updated_at	timestamp	Y			Last update time.
updated_by	varchar(100)	Y			Staff user id, stored as text.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.
deleted_by	varchar(100)	Y			Staff user id, stored as text.

imprints

MARC 264 second-indicator vocabulary: sequence of the imprint statement (earliest, intervening, current).

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 3 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
name	varchar(255)	Y			
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marc_class_sys

Classification systems a record can use: Library of Congress (LOC) or Dewey Decimal (DDC).

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 2 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
code	varchar(255)	Y			
name	varchar(255)	Y			
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marc_templates

Cataloguing templates: the set of MARC fields pre-filled in the record editor for a material type (e.g. BOOKS_LC_PCAARRD).

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 7 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
name	varchar(255)	Y			Template name, e.g. BOOKS_LC_PCAARRD.
fields	text	Y			The template's MARC fields and default values (serialised).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marc_templates_backup

A copy of marc_templates kept as a backup. Not read or written by the application.

Status: Unused · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
name	varchar(255)	Y			Template name.
fields	text	Y			Serialised fields.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marctag_indicators_loc

Allowed MARC indicator values. Referenced by id from inside catalogs.field_data.

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 312 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.

Column	Type	Null	Default	Key	Description
indicator	varchar(255)	N	''		Indicator value.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marctag_subfields_loc

MARC 21 subfields per tag (e.g. 245$a Title). Referenced by id from inside catalogs.field_data.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 2,606 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
marctags_loc_id	bigint unsigned	Y			Parent tag (marctags_loc.id; implicit).
code	varchar(255)	N			Subfield code, e.g. 245$a or LDR$06.
description	varchar(255)	N			Subfield name.
repeatable	tinyint	N	0		1 if the subfield may repeat.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

marctags_loc

The MARC 21 bibliographic tag list (Library of Congress): tag code, name, indicator descriptions and repeatability. Referenced by id from inside catalogs.field_data.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 249 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
code	varchar(255)	N			MARC tag (e.g. 245) or LDR.
description	varchar(255)	N			Tag name.
first_indictors	varchar(255)	Y			First-indicator definition (column name misspelt in the schema).
second_indicators	varchar(255)	Y			Second-indicator definition.
Repeatable	tinyint	N	0		1 if the tag may repeat.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

material_type_maps

Rules that map a value in an imported record (e.g. Koha item type in 942$c) to a local material type.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 1 · **Primary key:** id · **Other indexes:** UQ (name)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.

Column	Type	Null	Default	Key	Description
name	varchar(100)	N		UQ	Map name (unique), e.g. 'Koha - FNRI'.
source_field	varchar(20)	N			MARC field read, e.g. 942$c.
mappings	json	N			JSON: source value -> materials.id.
default_material_type_id	bigint unsigned	Y			Fallback material type.
created_by	bigint unsigned	Y			Staff account that created the row.
updated_by	bigint unsigned	Y			Staff account that updated the row.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

materials

Material types (Books, Serials, Thesis/Dissertations, ...). is_marc_type marks types that correspond to a MARC record type; is_active hides retired types.

Status: Reference · **Storage:** InnoDB, latin1_swedish_ci · **Rows:** 25 · **Primary key:** id · **Other indexes:** UQ (name)

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
name	varchar(100)	N		UQ	Display name (unique).
code	varchar(100)	Y			MARC leader 06 code for MARC types, '#' otherwise.
tooltip	text	Y			Help text shown in the editor.
is_active	tinyint(1)	N	1		0 hides the type from new records.
is_marc_type	tinyint(1)	N	0		1 = maps to a MARC record type.
created_by	bigint unsigned	Y			Staff account that created the row.
updated_by	bigint unsigned	Y			Staff account that updated the row.
created_at	timestamp	N	now()		Row creation time.
updated_at	timestamp	N	now()		Last update time.

series

Series statements of the legacy holdings model.

Status: Legacy · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
description	varchar(255)	Y			
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.

type_of_imprints

MARC 264 function vocabulary used by the editor (Production, Publication).

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 2 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
name	varchar(255)	Y			
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

view_catalogs

A base table (not a SQL view, despite its name) holding a flattened projection of each catalog record: title, authors, subjects, publisher, year, call number and so on, extracted from the MARC JSON. It is written by the cataloguing code whenever a record changes, is the Laravel Scout model indexed into Meilisearch, and is what the OPAC lists, filters and displays.

Status: Derived · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 34 · **Primary key:** id · **Other indexes:** IX (pub_year_num)

Column	Type	Null	Default	Key	Description
id	int unsigned	N		PK	Surrogate primary key.
catalog_id	int unsigned	N	0		The catalog record this row projects (catalogs.id; implicit 1:1). Not indexed.
isbn	varchar(100)	Y			ISBN (MARC 020$a).
issn	varchar(100)	Y			ISSN (MARC 022$a).
accession_number	varchar(255)	Y			Copied from catalogs.
is_published	tinyint	N	0		Copied from catalogs; filtered on by the OPAC.
catalog_import_id	tinyint	Y			Copied from catalogs.
material_type_id	int	Y			Material type id (copied).
material_type	varchar(100)	Y			Material type name (denormalised).
series_statement_id	int	Y			Legacy series id.
series_description	varchar(255)	Y			Series statement (MARC 490/830).
title_statement	text	Y			Title statement (MARC 245), may contain the statement of responsibility.
summary	text	Y			Summary/abstract (MARC 520).
general_note	text	Y			General note (MARC 500).
call_number	varchar(50)	Y			Call number (MARC 050/082/090).
catalog_date	varchar(50)	Y			Cataloguing date as text.
catalog_status	varchar(50)	Y			'Cataloged' or 'Uncataloged'.
date_acquired	date	Y			Date the material was acquired.
edition	varchar(100)	Y			Edition statement (MARC 250).
joint_authors	text	Y			All authors, ' / '-separated (MARC 1XX/7XX).
subjects	json	Y			Subject terms as a JSON array (MARC 6XX).

Column	Type	Null	Default	Key	Description
publisher	text	Y			Publisher (MARC 260/264$b).
pub_url	text	Y			Electronic location (MARC 856$u).
pub_year	varchar(50)	Y			Publication year as written (e.g. '[2009]').
pub_year_num	smallint unsigned	Y		IX	Numeric publication year for sorting and range filters (indexed).
created_at	timestamp	Y			Row creation time.
created_by	int	Y			Staff user id.
updated_at	timestamp	Y			Last update time.
updated_by	int	Y			Staff user id.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.
deleted_by	int	Y			Staff user id.

A.2 Holdings and digital objects

copies

Physical copies (items) of a title, identified by circulation_number (barcode). Status drives availability for reservation and loan.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** IX (catalog_id)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
catalog_id	varchar(50)	Y		IX	The title (catalogs.id; implicit; stored as text). Index named holding_id.
title	text	Y			Copy of the title (denormalised).
circulation_number	varchar(50)	Y			Copy barcode; used as the natural key by circulation. Not unique, not indexed.
cost	int	N	0		Acquisition cost (whole units).
acquisition_mode	int	Y			How it was acquired (code).
source	varchar(50)	Y			Supplier or donor.
location	char(50)	Y	1		Shelving location code (default 1).
status	char(50)	Y	4		copies_status.order_no as text (default '4' Not Available).
series_statement_id	int	Y			Legacy series id.
presented	timestamp	Y			When the copy was last confirmed during an inventory count.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

frontpages

The cover image of a record (front and back paths). Generated covers are written here too.

Status: Core · **Storage:** InnoDB, latin1_swedish_ci · **Rows:** 34 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
`id`	int	N		PK	Surrogate primary key.
`catalog_id`	varchar(24)	N			The record (catalogs.id; implicit; text).
`AcquisitionID`	varchar(24)	N			Legacy acquisition id.
`FrontPageName`	varchar(250)	N			File name.
`FrontPageType`	varchar(250)	N			MIME type.
`FrontPageLocation`	varchar(255)	Y			Front-cover path under public/.
`FrontPageLocation-back`	varchar(250)	N			Back-cover path (hyphenated column name).
`created_at`	timestamp	Y	now()		Row creation time.
`updated_at`	timestamp	Y			Last update time.
`deleted_at`	timestamp	Y			Soft-delete time; NULL means active.

fullpages

Full-page image file names from an earlier design. Not used by the current code.

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`filename`	varchar(50)	Y			
`created_at`	timestamp	Y			Row creation time.
`updated_at`	timestamp	Y			Last update time.

holdingscopypresenteds

Inventory (physical count) log: copies confirmed present during a stock-taking, by catalog and circulation number.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`catalog_id`	varchar(100)	Y			The record.
`circulation_number`	varchar(100)	Y			The copy confirmed present.
`created_at`	timestamp	Y			Row creation time.
`updated_at`	timestamp	Y			Last update time.

multimedia

The full-text file attached to a record (normally a PDF): file name, storage path under public/, type and description. Read through the guarded /read route and the download route.

Status: Core · **Storage:** InnoDB, latin1_swedish_ci · **Rows:** 4 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
catalog_id	varchar(24)	Y			The record (catalogs.id; implicit; text).
FileName	varchar(255)	Y			Original file name.
FileLocation	varchar(255)	Y			Path under public/ (e.g. pdf/....pdf).
FileType	varchar(255)	Y			MIME type.
Extension	varchar(255)	Y			File extension.
Permission	varchar(255)	Y			Access note.
Description	varchar(255)	Y			Description.
created_at	timestamp	Y	now()		Row creation time.
updated_at	timestamp	Y	now()		Last update time.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.

p_d_f_s

PDF file names per catalog from an earlier design (Laravel's table name for the PDF model). Superseded by multimedia.

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
catalog_id	int	Y			The record.
filename	varchar(255)	Y			File name.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

thumbnails

Thumbnail file names from an earlier design. Not used by the current code.

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
filename	varchar(255)	Y			
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

view_total_copies

A base table holding copy totals per legacy holdings id. Not read or written by the application.

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** none · **Other indexes:** none

Column	Type	Null	Default	Key	Description
total	bigint	Y			Copy count.
holdings_id	varchar(24)	Y			Legacy holdings id.

A.3 Circulation

circulation_fine_history

Fine assessment for a transaction (one row per transaction_code): overdue amount and hours, damage fine and its type, notes. Created empty with the reservation and filled at check-in.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** UQ (id)

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
transaction_code	varchar(50)	Y			Links to circulations.transaction_code.
overdue_fine_amount	varchar(50)	Y			Overdue fine (text).
overdue_total_hours	varchar(50)	Y			Hours overdue (text).
other_fine_amount	varchar(50)	Y			Damage fine (text).
circulation_fine_id	int	Y			Damage type (circulation_fines.id).
other_fine_description	varchar(255)	Y			Damage description.
fine_note	varchar(255)	Y			Staff note.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

circulation_fines

Damage types that can incur a fine (Water Damage, Page Tears, ...).

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 10 · **Primary key:** id · **Other indexes:** UQ (id)

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
name	varchar(50)	Y			Damage type.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

circulation_reservations

The patron's reservation request details (requested visit date and time, title) linked to circulations by reservation_code.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
user_id	int	Y			The patron.
location_id	int	Y			Pickup location (always 1).
requested_by	varchar(100)	Y			Patron's name (copied).
title_statement	text	Y			Title requested (copied).
reservation_code	varchar(50)	Y			Links to circulations.reservation_code.
visit_date	varchar(50)	Y			Requested visit date (text).
visit_time	varchar(50)	Y			Requested visit time (text).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

circulation_status

Circulation lifecycle states. circulations.status stores order_no (not id): 1 For Approval, 2 Approved, 3 Open (checked out), 4 Declined, 5 Closed.

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 5 · **Primary key:** id · **Other indexes:** IX (order_no)

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
name	varchar(50)	Y			State name.
order_no	int	Y		IX	The code stored in circulations.status.
description	varchar(255)	Y			Meaning of the state.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

circulations

One row per circulation transaction: a reservation that may become a loan. Carries the reservation and transaction codes, the copy, the status (see circulation_status.order_no), due date, fines and who approved, declined, released and received it.

Status: Core · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** UQ (id)

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
user_id	int	Y			The patron (users.id; implicit).
copy_id	int	Y			The copy (copies.id); set at check-out.
catalog_id	int	Y			The title (catalogs.id; implicit).

Column	Type	Null	Default	Key	Description
reservation_code	varchar(50)	Y			Code shown to the patron; links reservation, messages and chat. Not indexed.
transaction_code	varchar(50)	Y			Links the fine assessment (circulation_fine_history). Not indexed.
status	int	N	1		Lifecycle state = circulation_status.order_no (1-5).
due_date	datetime	Y			When the loan is due.
overdue_fine_rate	varchar(50)	Y			Overdue fine per hour (stored as text).
waived_amount	varchar(50)	Y			Fine waived at check-in (text).
total_fine	varchar(50)	Y			Total fine assessed (text).
total_fine_paid	varchar(50)	Y			Amount paid at check-in (text).
user_library_code	varchar(50)	Y			Patron card code, copied from users.library_code.
circulation_number	varchar(50)	Y			The copy's barcode, assigned at approval.
approved_at	datetime	Y			When staff approved.
approved_by	varchar(50)	Y			Approving staff member's name (not id).
declined_at	datetime	Y			When declined.
declined_by	varchar(50)	Y			Declining staff name.
released_at	datetime	Y			When checked out.
released_by	varchar(50)	Y			Staff name.
received_at	datetime	Y			When returned (checked in).
received_by	varchar(50)	Y			Staff name.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

copies_status

Copy states. copies.status stores order_no: 1 Available, 2 Reserved, 3 Borrowed, 4 Not Available, 5 Deleted, 6 Lost, 7 For Repair.

Status: Reference · **Storage:** InnoDB, utf8mb4_general_ci · **Rows:** 7 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
order_no	varchar(50)	Y			The code stored in copies.status.
name	varchar(50)	Y			State name.
description	varchar(255)	Y			Meaning of the state.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

reservation_messages

Chat thread between staff and the patron about one reservation, keyed by reservation_code, with read receipts (seen_at).

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** IX (reservation_code)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
reservation_code	varchar(125)	N		IX	The reservation the thread belongs to (indexed).
user_id	bigint unsigned	Y			The patron who owns the thread.
sender_id	bigint unsigned	Y			Who wrote the message (users.id).
sender_role	varchar(20)	N	admin		'admin' or 'patron'.
message	text	N			Message text.
seen_at	timestamp	Y			When the recipient read it.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

A.4 People, organisations and access control

consortias

The 16 regional R&D consortia of the PCAARRD network (CorCAARRD, MAARRDEC, ...). Referenced only by the legacy borrowing model.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 16 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
name	varchar(25)	Y			Consortium acronym.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

model_has_permissions

Permissions granted directly to a user, outside roles (Spatie pivot). Currently empty.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** permission_id, model_id, model_type · **Other indexes:** IX (model_id, model_type)

Column	Type	Null	Default	Key	Description
permission_id	bigint unsigned	N		PK FK	permissions.id (declared FK).
model_type	varchar(125)	N		PK	Owning model class.
model_id	bigint unsigned	N		PK	users.id (polymorphic).

model_has_roles

Which roles each user has (Spatie polymorphic pivot: model_type = App\Models\User, model_id = users.id).

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 3 · **Primary key:** role_id, model_id, model_type · **Other indexes:** IX (model_id, model_type)

Column	Type	Null	Default	Key	Description
role_id	bigint unsigned	N		PK FK	roles.id (declared FK).
model_type	varchar(125)	N		PK	Owning model class (App\Models\User).
model_id	bigint unsigned	N		PK	users.id (polymorphic; no FK).

password_resets

Laravel password-reset tokens (hashed), one per email.

Status: Framework · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** none · **Other indexes:** none

Column	Type	Null	Default	Key	Description
email	varchar(255)	N			Account email.
token	varchar(255)	N			Hashed reset token.
created_at	timestamp	Y			Row creation time.

patrontypes

Patron categories (Student, Researcher, Farmer, ...), used for profiling and reports.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 8 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
types	varchar(50)	N			Category name.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

permissions

Permission names checked by the application (access.dashboard, create.catalog, ...).

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 35 · **Primary key:** id · **Other indexes:** UQ (name, guard_name)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
name	varchar(125)	N		IX	Permission key checked in code (e.g. access.dashboard).
guard_name	varchar(125)	N			Auth guard ('web').
display_name	varchar(125)	Y			Label.

Column	Type	Null	Default	Key	Description
description	varchar(125)	Y			Description.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

personal_access_tokens

Laravel Sanctum API tokens. Present by default; unused.

Status: Framework · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** UQ (token); IX (tokenable_type, tokenable_id)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
tokenable_type	varchar(125)	N		IX	Token owner class.
tokenable_id	bigint unsigned	N			Token owner id.
name	varchar(125)	N			Token name.
token	varchar(64)	N		UQ	Hashed token (unique).
abilities	text	Y			Scopes.
last_used_at	timestamp	Y			Last use.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

regions

Philippine regions (PSGC codes) for patron location. The only MyISAM table.

Status: Reference · **Storage:** MyISAM, utf8mb3_general_ci · **Rows:** 17 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int	N		PK	Surrogate primary key.
psgcCode	varchar(255)	Y			Philippine Standard Geographic Code.
desc	text	Y			Region name.
regCode	varchar(255)	Y			Region code.

role_has_permissions

Which permissions each role grants (Spatie pivot; declared foreign keys).

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 124 · **Primary key:** permission_id, role_id · **Other indexes:** IX (role_id)

Column	Type	Null	Default	Key	Description
permission_id	bigint unsigned	N		PK FK	permissions.id (declared FK).
role_id	bigint unsigned	N		PK FK	roles.id (declared FK).

roles

Staff roles (Spatie laravel-permission): super_admin, cataloging_admin, circulation_admin, serial_admin, system_admin, faculty, developer.

Status: Reference · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 7 · **Primary key:** id · **Other indexes:** UQ (name, guard_name)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
name	varchar(125)	N		IX	Role key.
guard_name	varchar(125)	N			Auth guard ('web').
display_name	varchar(125)	Y			Label.
description	varchar(125)	Y			Description.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

users

All accounts: patrons and staff. usertype separates patrons from staff; staff permissions come from Spatie roles. Holds profile data (personal information), the bcrypt password hash, the patron library card code and the ban/block flags.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 5 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
username	varchar(100)	Y			Optional login name (staff).
name	varchar(255)	N			Full name.
email	varchar(255)	N			Login email. Not unique at database level (validated in code).
contactnum	varchar(50)	Y			Contact number.
usertype	enum('Administrator','Student','Patron','Developer')	Y	Patron		Account type: Patron, Administrator, Developer ('Student' unused).
is_banned	tinyint(1)	N	0		1 = cannot log in or read (set by staff).
is_blocked	tinyint(1)	N	0		1 = cannot use chat/inquiries.
birthdate	varchar(50)	Y			Date of birth as text.
location	text	Y			Address/location.
region_id	varchar(10)	Y			Region (regions.id; implicit; text).
position1	varchar(100)	Y			Position/designation (1 of 4 slots).
position2	varchar(100)	Y			Position (slot 2).
position3	varchar(100)	Y			Position (slot 3).
position4	varchar(100)	Y			Position (slot 4).

Column	Type	Null	Default	Key	Description
sex	enum('Female','Male')	Y			Female/Male.
affiliation	varchar(255)	Y			Organisation.
email_verified_at	timestamp	Y			Email verification time.
password	varchar(255)	N			bcrypt hash.
remember_token	varchar(100)	Y			'Remember me' token.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.
backup	int	Y			Legacy migration flag; unused.
patrontypes_id	bigint unsigned	N			Patron category (patrontypes.id; implicit). Staff accounts use 0.
library_code	varchar(255)	Y			Patron library card code (generated at registration).

A.5 Engagement and discovery

downloads

Append-only log of full-text downloads by patrons. Also makes the material appear in the patron's To Review list.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 6 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
catalog_id	varchar(50)	Y			Record (catalogs.id; implicit; text).
user_id	int	Y			Patron (users.id; implicit).
to_review	date	Y			Legacy review-due date; unused.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

ratings

Patron reviews: a 1-5 star rating and optional remarks per material. The application allows one review per patron per material.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 4 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
catalog_id	varchar(25)	Y			Record (catalogs.id; implicit; VARCHAR(25)).
user_id	int	Y			Reviewer (users.id; implicit).
rating	int	Y			Stars, 1-5 (enforced in code only).

Column	Type	Null	Default	Key	Description
remarks	text	Y			Review text (optional).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

reads

First time a patron opens a material in the online reader; unique per (user, catalog). Feeds To Review and the dashboard's Online reads.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 2 · **Primary key:** id · **Other indexes:** UQ (user_id, catalog_id)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
user_id	bigint unsigned	N		IX	Patron (users.id; implicit).
catalog_id	bigint unsigned	N			Record (catalogs.id; implicit).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

saved_records

Materials a patron saved to their list; unique per (user, catalog), with declared foreign keys.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 11 · **Primary key:** id · **Other indexes:** IX (catalog_id); UQ (user_id, catalog_id)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
user_id	bigint unsigned	N		IX FK	Patron (declared FK to users).
catalog_id	int unsigned	N		IX FK	Record (declared FK to catalogs).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

view_most_downloaded (SQL view)

SQL view: the 10 most downloaded non-deleted records.

Status: Derived · **Columns:** 2 · **Rows:** 4

Column	Type	Null	Default	Key	Description
total	bigint	N	0		Derived; see the view definition (Appendix C).
catalog_id	varchar(50)	Y			Derived; see the view definition (Appendix C).

view_most_viewed (SQL view)

SQL view: the 5 most viewed non-deleted records.

Status: Derived · **Columns:** 2 · **Rows:** 5

Column	Type	Null	Default	Key	Description
total	bigint	N	0		Derived; see the view definition (Appendix C).
catalog_id	varchar(50)	Y			Derived; see the view definition (Appendix C).

view_ratings (SQL view)

SQL view: ratings joined with the reviewer's name; used for record pages and averages.

Status: Derived · **Columns:** 8 · **Rows:** 4

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N	0		Identifier.
catalog_id	varchar(25)	Y			Derived; see the view definition (Appendix C).
user_id	int	Y			Derived; see the view definition (Appendix C).
rating	int	Y			Derived; see the view definition (Appendix C).
remarks	text	Y			Derived; see the view definition (Appendix C).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
name	varchar(255)	Y			Derived; see the view definition (Appendix C).

view_searches (SQL view)

SQL view over the legacy holdings table (title + summary per record). Returns no rows because holdings is empty; search now runs on Meilisearch.

Status: Legacy · **Columns:** 4 · **Rows:** 0

Column	Type	Null	Default	Key	Description
id	int	N	0		Identifier.
search	mediumtext	Y			Derived; see the view definition (Appendix C).
material_type_id	int	Y			Derived; see the view definition (Appendix C).
material	varchar(100)	Y			Derived; see the view definition (Appendix C).

view_to_reviews (SQL view)

SQL view: downloads joined with the legacy holdings table for the title; the title is therefore always NULL. Superseded by reviewPendingQuery().

Status: Legacy · **Columns:** 7 · **Rows:** 6

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N	0		Identifier.
catalog_id	varchar(50)	Y			Derived; see the view definition (Appendix C).

Column	Type	Null	Default	Key	Description
user_id	int	Y			Derived; see the view definition (Appendix C).
to_review	date	Y			Derived; see the view definition (Appendix C).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
title_statement	varchar(255)	Y			Derived; see the view definition (Appendix C).

views

Append-only log of record views (opening a record page): who (or guest), which record, IP address and user agent. Feeds view counts, the dashboard and 'most viewed'.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 79 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
is_guest	enum('YES','NO')	Y			'YES' for anonymous visitors, 'NO' when signed in.
user_id	int	Y			Viewer (users.id; implicit; NULL for guests).
catalog_id	varchar(50)	Y			Record viewed (catalogs.id; implicit; text).
ip_view	varchar(255)	Y			Viewer's IP address.
user_agent	varchar(512)	Y			Browser user agent (first 512 characters).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

A.6 Communication and feedback

csfs

Client Satisfaction Feedback (government CSF form): seven dimension ratings (responsiveness, reliability, access and facilities, communication, integrity, assurance, outcome) and three comments.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
user_id	bigint unsigned	N			Respondent.
rating1	varchar(255)	Y			Responsiveness.
rating2	varchar(255)	Y			Reliability.
rating3	varchar(255)	Y			Access and facilities.
rating4	varchar(255)	Y			Communication.
rating5	varchar(255)	Y			Integrity.
rating6	varchar(255)	Y			Assurance.
rating7	varchar(255)	Y			Outcome.
comment1	varchar(50)	Y			Comment 1.

Column	Type	Null	Default	Key	Description
comment2	varchar(50)	Y			Comment 2.
comment3	varchar(50)	Y			Comment 3.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

inquiries

Questions a registered patron sends about a material or the library.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
user_id	int	Y			Patron (users.id; implicit).
fullname	varchar(255)	Y			Name (copied).
contactnum	varchar(255)	Y			Contact number (copied).
email	varchar(255)	Y			Email (copied).
catalog_id	varchar(50)	Y			Record asked about (optional).
subject	varchar(125)	Y			Subject line.
message	text	Y			First message.
guest_replied_at	timestamp	Y			Added for guest inquiries; unused since inquiries became registered-only.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

inquiry_seens

Per-staff read receipt for each inquiry (unique per inquiry and user); drives the unread badges.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** IX (inquiry_id); UQ (inquiry_id, user_id)

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
inquiry_id	bigint unsigned	N		IX	Inquiry (indexed).
user_id	bigint unsigned	N			Staff member.
seen_at	timestamp	Y			When they last opened the thread.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

news

News items and announcements shown on the OPAC (title, body, cover, link, featured flag).

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
news_title	text	Y			Headline.
redirect_url	text	Y			Optional external link.
news_body	text	Y			Body (HTML).
featured	enum('Yes','No')	N	No		'Yes' to feature on the home page.
news_cover	varchar(255)	Y			Cover image path.
created_by	varchar(50)	Y			Author (staff name).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
deleted_at	timestamp	Y			Soft-delete time; NULL means active.

reply_inquiries

Messages in an inquiry thread, from staff or the patron, with when and by whom each was seen.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
inquiry_id	int	Y			Thread (inquiries.id; implicit).
user_id	int	Y			Sender (users.id).
message	text	Y			Message text.
seen	timestamp	Y			When the other side read it.
seen_by	int	Y			Who read it.
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

view_inquiries (SQL view)

SQL view: inquiries with message counts and unread-reply counts. Its title column comes from legacy holdings and is always NULL.

Status: Derived · **Columns:** 14 · **Rows:** 0

Column	Type	Null	Default	Key	Description
id	bigint unsigned	Y	0		Identifier.
user_id	int	Y			Derived; see the view definition (Appendix C).
fullname	varchar(255)	Y			Derived; see the view definition (Appendix C).
contactnum	varchar(255)	Y			Derived; see the view definition (Appendix C).

Column	Type	Null	Default	Key	Description
email	varchar(255)	Y			Derived; see the view definition (Appendix C).
catalog_id	varchar(50)	Y			Derived; see the view definition (Appendix C).
subject	varchar(125)	Y			Derived; see the view definition (Appendix C).
message	text	Y			Derived; see the view definition (Appendix C).
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.
title_statement	varchar(255)	Y			Derived; see the view definition (Appendix C).
created_at_string	varchar(82)	Y			Derived; see the view definition (Appendix C).
total_msg	bigint	Y			Derived; see the view definition (Appendix C).
new_reply	bigint	Y			Derived; see the view definition (Appendix C).

view_inquiry_threads (SQL view)

SQL view: each reply with the inquiry owner and the replier's name.

Status: Derived · **Columns:** 8 · **Rows:** 0

Column	Type	Null	Default	Key	Description
id	bigint unsigned	Y	0		Identifier.
user_id	int	Y			Derived; see the view definition (Appendix C).
replied_by	int	Y			Derived; see the view definition (Appendix C).
replied_by_name	varchar(255)	Y			Derived; see the view definition (Appendix C).
replied_msg	text	Y			Derived; see the view definition (Appendix C).
seen	timestamp	Y			Derived; see the view definition (Appendix C).
created_at_string	varchar(82)	Y			Derived; see the view definition (Appendix C).
thread_date	varchar(82)	Y			Derived; see the view definition (Appendix C).

visits

Library visit schedules from an earlier design. No primary key; not written by the current code.

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** none · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N			Identifier (not a primary key).
user_id	int	Y			Visitor.
fullname	varchar(100)	Y			Name.
contactnum	varchar(20)	Y			Contact number.
email	varchar(100)	Y			Email.
schedule	datetime	Y			Scheduled visit.
location	varchar(100)	Y			Venue.
site_visit	int	Y			Visit type code.

Column	Type	Null	Default	Key	Description
created_at	timestamp	Y			Row creation time.
updated_at	timestamp	Y			Last update time.

A.7 Integration, configuration and framework

failed_jobs

Laravel failed queue jobs. Unused for the same reason.

Status: Framework · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
uuid	varchar(255)	N			Job id.
connection	text	N			Connection.
queue	text	N			Queue.
payload	longtext	N			Serialised job.
exception	longtext	N			Error.
failed_at	timestamp	N	now()		Failure time.

jobs

Laravel queue (database driver). Unused: the queue runs synchronously.

Status: Framework · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	bigint unsigned	N		PK	Surrogate primary key.
queue	varchar(255)	N			Queue name.
payload	longtext	N			Serialised job.
attempts	tinyint unsigned	N			Attempts.
reserved_at	int unsigned	Y			Reserved time (epoch).
available_at	int unsigned	N			Available time (epoch).
created_at	int unsigned	N			Created (epoch).

migrations

Laravel's record of which schema migrations have run.

Status: Framework · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 18 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
id	int unsigned	N		PK	Surrogate primary key.
migration	varchar(125)	N			Migration file name.
batch	int	N			Run batch.

rag_sync_queue

Outbox for the AI (RAG) knowledge base: one row per catalog change to push to the Hermes RAG service (upsert or delete), with retry count, last error and processed time.

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 34 · **Primary key:** id · **Other indexes:** IX (catalog_id); IX (processed_at)

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`catalog_id`	bigint unsigned	N		IX	Record to sync (catalogs.id; indexed).
`action`	varchar(16)	N			'upsert' or 'delete'.
`reason`	varchar(32)	N			Why it was queued (created, updated, deleted, manual, ...).
`content_hash`	varchar(64)	Y			Hash of the content last sent, to skip unchanged records.
`attempts`	int unsigned	N	0		Delivery attempts.
`error`	text	Y			Last error.
`created_at`	timestamp	Y			Row creation time.
`processed_at`	timestamp	Y		IX	When delivered; NULL = pending (indexed).

settings

Application settings as key/value pairs (AI mode switch, placeholder covers, SEO options).

Status: Core · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 4 · **Primary key:** id · **Other indexes:** UQ (key)

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`key`	varchar(191)	N		UQ	Setting name (unique).
`value`	text	Y			Setting value (text).
`created_at`	timestamp	Y			Row creation time.
`updated_at`	timestamp	Y			Last update time.

A.8 Legacy (pre-MARC) structures

authors

Personal authors of legacy holdings records (MARC 100/700-like attributes). Empty; authors now live in the MARC JSON.

Status: Legacy · **Storage:** InnoDB, latin1_swedish_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Columns (17): `id` int, `catalog_id` varchar(24), `AuthorTag` int, `author_name` varchar(255), `author_name2` varchar(255), `numeration` varchar(100), `title_and_words` varchar(255), `relator_term` varchar(255), `date_associated` date, `fuller_form` varchar(255), `affiliation` text, `CreatedBy` varchar(255), `created_at` timestamp, `UpdatedBy` varchar(255), `updated_at` timestamp, `IsActive` tinyint(1), `IsDeleted` tinyint(1).

barrowed_books

The earlier borrowing model (sic: 'barrowed'), with its own status enum. Empty; superseded by circulations.

Status: Legacy · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Columns (15): `id` bigint unsigned, `user_id` int, `fullname` varchar(255), `contactnum` varchar(50), `email` varchar(100), `catalog_id` varchar(50), `consortia_id` int, `status` enum('For Approval','Borrowed','Returned'), `approved_at` timestamp, `approved_by` varchar(100), `update_return_by` varchar(100), `returned_at` timestamp, `created_at` timestamp, `updated_at` timestamp, `deleted_at` timestamp.

corporate_authors

Corporate authors of legacy records (MARC 110/710-like). Empty.

Status: Legacy · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`catalog_id`	varchar(50)	Y			
`author_type`	int	Y			
`author_name`	varchar(255)	Y			
`subordinate_unit`	varchar(255)	Y			
`location_meeting`	varchar(255)	Y			
`date_of_meeting`	date	Y			
`number_of_part`	varchar(255)	Y			
`created_at`	timestamp	Y			Row creation time.
`updated_at`	timestamp	Y			Last update time.
`isActive`	tinyint(1)	Y			
`isDeleted`	tinyint(1)	Y			

for_reviews

Empty table with only id and timestamps; superseded by reviewPendingQuery().

Status: Unused · **Storage:** InnoDB, utf8mb4_unicode_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Column	Type	Null	Default	Key	Description
`id`	bigint unsigned	N		PK	Surrogate primary key.
`created_at`	timestamp	Y			Row creation time.
`updated_at`	timestamp	Y			Last update time.

holdings

The pre-MARC bibliographic model: 89 flat columns per title (title, edition, physical description, copy data, ...). Empty; superseded by catalogs (MARC JSON) and copies, but still referenced by several SQL views.

Status: Legacy · **Storage:** InnoDB, latin1_swedish_ci · **Rows:** 0 · **Primary key:** id · **Other indexes:** none

Columns (89): `id` int, `catalog_id` varchar(24), `call_number` varchar(24), `catalog_date` varchar(50), `CatalogSource` varchar(50), `material_type_id` int, `ISBN` varchar(50), `ISSN` varchar(50), `CallNumber` varchar(255), `CallNumberFull` varchar(255), `title_statement` varchar(255), `edition_statement` varchar(255), `edition_statement_remainder` varchar(255), `PhysicalDescriptionEtc` varchar(255), `MediaTypeCode` varchar(50), `MediaSource` varchar(50), `CarrierSource` varchar(50), `series_statement_id` int, `series_statement` varchar(150), `general_note` text, `bibliography` varchar(255), `FormattedContents` varchar(200), `ScopeandContent` varchar(150), `summary` text, `AdditionalPhysicalFormAvailableNote` varchar(150), `PublicationStatus` tinyint, `IsActive` tinyint(1), `deleted` tinyint(1), `deleted_at` timestamp, `CreatedBy` varchar(255), `created_at` timestamp, `UpdatedBy` varchar(255), `updated_at` datetime, `LanguageID` int, `Edition` varchar(100), `VolumeNo` varchar(50), `BroadClass` varchar(10), `ContentTypeID` int, `ContentTypeSource` varchar(50), `VolumeSequentialDesignation` varchar(150), `PhysicalFormAvailableNote` varchar(150), `AvailabilitySource` varchar(150), `AvailabilityConditions` varchar(150), `MediaTypeID` int, `CarrierTypeID` int, `CarrierTypeCode` int, `ContentTypeCode` int, `DissertationNote` varchar(500), `AbbreviatedTitle` varchar(255), `SoundCharacteristics` varchar(255), `DatesofPublication` varchar(255), `TimePeriodofCreation` varchar(255), `CitationReferencesNote` varchar(255), `SpecialCodedDates` varchar(50), `ProductionDistributionandYearSubmitted` varchar(500), `RestrictiononAccessNote` varchar(200), `TypesofReportandPeriodCoveredNote` varchar(200), `consortia_id` int, `accession_number` varchar(50), `circulation_number` varchar(50), `cost_price` decimal(10,2), `acquisition_modes` enum('Purchase','Subscription','Donation','Complimentary','Exchange','Download'), `copy_number` char(10), `classifications_system` enum('Library of Congress','Dewey Decimal Classification'), `date_acquired` date, `source` varchar(50), `user_restriction` varchar(25), `item_status` varchar(50), `temporary_location` text, `nonpublic_note` varchar(255), `frequency` enum('Daily','Weekly','Monthly','Quarterly','Yearly','Irregular'), `volume` varchar(25), `issue_date` date, `issue_number` varchar(50), `publication_date` varchar(20), `classification_number` varchar(50), `author_number` varchar(100), `copyright_date` date, `title_remainder` varchar(255), `statement_responsibility` text, `extent` text, `other_physical` text, `dimension` text, `imprint` enum('Not Applicable/Not Provided/Earliest','Intervening','Current/Last'), `type_of_imprint` enum('Production','Publication'), `publication_place` text, `publisher` text, `copies` int, `pub_url` text.

view_borrowed_books (SQL view)

SQL view over barrowed_books with the title from holdings and a 'total days' figure. Contains a NULL-comparison defect (see Analysis).

Status: Legacy · **Columns:** 19 · **Rows:** 0

Columns (19): `id` bigint unsigned, `user_id` int, `fullname` varchar(255), `contactnum` varchar(50), `email` varchar(100), `catalog_id` varchar(50), `consortia_id` int, `status` enum('For Approval','Borrowed','Returned'), `approved_at` timestamp, `approved_by` varchar(100), `returned_at` timestamp, `created_at` timestamp, `updated_at` timestamp, `deleted_at` timestamp, `title_statement` varchar(255), `request_at_string` varchar(82), `total_days` int, `approved_at_string` varchar(82), `returned_at_string` varchar(82).

view_holding_frontpages (SQL view)

SQL view: legacy holdings that have a cover image.

Status: Legacy · **Columns:** 4 · **Rows:** 0

Column	Type	Null	Default	Key	Description
`id`	int	N	0		Identifier.
`catalog_id`	varchar(24)	N			Derived; see the view definition (Appendix C).

Column	Type	Null	Default	Key	Description
material_type_id	int	Y			Derived; see the view definition (Appendix C).
FrontPageLocation	varchar(255)	Y			Derived; see the view definition (Appendix C).

view_holdings (SQL view)

SQL view over legacy holdings with joined material, series and author list.

Status: Legacy · **Columns:** 23 · **Rows:** 0

Columns (23): id int, catalog_id int, circulation_number varchar(50), accession_number varchar(50), title_statement varchar(255), summary text, material_type_id int, series_statement_id int, catalog_date varchar(50), date_acquired date, edition varchar(100), created_at varchar(82), CreatedBy varchar(255), catalog_status varchar(11), material varchar(100), series_description varchar(255), call_number varchar(24), publisher text, pub_url text, joint_authors text, general_note text, pub_year varchar(20), deleted_at timestamp.

view_holdings_uncatalog (SQL view)

SQL view exposing the 89 legacy holdings columns plus derived values.

Status: Legacy · **Columns:** 90 · **Rows:** 0

Columns (90): id int, catalog_id varchar(24), call_number varchar(24), catalog_date varchar(50), CatalogSource varchar(50), material_type_id int, ISBN varchar(50), ISSN varchar(50), CallNumber varchar(255), CallNumberFull varchar(255), title_statement varchar(255), edition_statement varchar(255), edition_statement_remainder varchar(255), PhysicalDescriptionEtc varchar(255), MediaTypeCode varchar(50), MediaSource varchar(50), CarrierSource varchar(50), series_statement_id int, series_statement varchar(150), general_note text, bibliography varchar(255), FormattedContents varchar(200), ScopeandContent varchar(150), Summary text, AdditionalPhysicalFormAvailableNote varchar(150), PublicationStatus tinyint, IsActive tinyint(1), deleted tinyint(1), deleted_at timestamp, CreatedBy varchar(255), created_at timestamp, UpdatedBy varchar(255), updated_at datetime, LanguageID int, Edition varchar(100), VolumeNo varchar(50), BroadClass varchar(10), ContentTypeID int, ContentTypeSource varchar(50), VolumeSequentialDesignation varchar(150), PhysicalFormAvailableNote varchar(150), AvailabilitySource varchar(150), AvailabilityConditions varchar(150), MediaTypeID int, CarrierTypeID int, CarrierTypeCode int, ContentTypeCode int, DissertationNote varchar(500), AbbreviatedTitle varchar(255), SoundCharacteristics varchar(255), DatesofPublication varchar(255), TimePeriodofCreation varchar(255), CitationReferencesNote varchar(255), SpecialCodedDates varchar(50), ProductionDistributionandYearSubmitted varchar(500), RestrictiononAccessNote varchar(200), TypesofReportandPeriodCoveredNote varchar(200), consortia_id int, accession_number varchar(50), circulation_number varchar(50), cost_price decimal(10,2), acquisition_modes enum('Purchase','Subscription','Donation','Complimentary','Exchange','Download'), copy_number char(10), classifications_system enum('Library of Congress','Dewey Decimal Classification'), date_acquired date, source varchar(50), user_restriction varchar(25), item_status varchar(50), temporary_location text, nonpublic_note varchar(255), frequency enum('Daily','Weekly','Monthly','Quarterly','Yearly','Irregular'), volume varchar(25), issue_date date, issue_number varchar(50), publication_date varchar(20), classification_number varchar(50), author_number varchar(100), copyright_date date, title_remainder varchar(255), statement_responsibility text, extent text, other_physical text, dimension text, imprint enum('Not Applicable/Not Provided/Earliest','Intervening','Current/Last'), type_of_imprint enum('Production','Publication'), publication_place text, publisher text, copies int, material_desc varchar(100), joint_authors text.

Appendix B Index catalogue

All 24 secondary indexes (primary keys are listed in Appendix A). Indexes named after a foreign key were created by Laravel when the key was declared.

Table 12. Secondary indexes.

Table	Index	Kind	Columns
circulation_fine_history	id	Unique	id
circulation_fines	id	Unique	id
circulation_status	order_no	Non-unique	order_no
circulations	id	Unique	id
copies	holding_id	Non-unique	catalog_id
inquiry_seens	inquiry_seens_inquiry_id_index	Non-unique	inquiry_id
inquiry_seens	inquiry_seens_inquiry_id_user_id_unique	Unique	inquiry_id, user_id
material_type_maps	material_type_maps_name_unique	Unique	name
materials	materials_name_unique	Unique	name
model_has_permissions	model_has_permissions_model_id_model_type_index	Non-unique	model_id, model_type
model_has_roles	model_has_roles_model_id_model_type_index	Non-unique	model_id, model_type
permissions	permissions_name_guard_name_unique	Unique	name, guard_name
personal_access_tokens	personal_access_tokens_token_unique	Unique	token
personal_access_tokens	personal_access_tokens_tokenable_type_tokenable_id_index	Non-unique	tokenable_type, tokenable_id
rag_sync_queue	rag_sync_queue_catalog_id_index	Non-unique	catalog_id
rag_sync_queue	rag_sync_queue_processed_at_index	Non-unique	processed_at
reads	reads_user_id_catalog_id_unique	Unique	user_id, catalog_id
reservation_messages	reservation_messages_reservation_code_index	Non-unique	reservation_code
role_has_permissions	role_has_permissions_role_id_foreign	Non-unique	role_id
roles	roles_name_guard_name_unique	Unique	name, guard_name
saved_records	saved_records_catalog_id_foreign	Non-unique	catalog_id
saved_records	saved_records_user_id_catalog_id_unique	Unique	user_id, catalog_id
settings	key	Unique	key
view_catalogs	view_catalogs_pub_year_num_index	Non-unique	pub_year_num

Declared foreign keys:

Table 13. Declared foreign keys.

Column	References
model_has_permissions.permission_id	permissions.id

Column	References
model_has_roles.role_id	roles.id
role_has_permissions.permission_id	permissions.id
role_has_permissions.role_id	roles.id
saved_records.catalog_id	catalogs.id
saved_records.user_id	users.id

Appendix C SQL view definitions

Definitions as stored by MySQL, with schema qualifiers and back-quotes removed for readability.

view_borrowed_books

```
select a.id AS id,a.user_id AS user_id,a.fullname AS fullname,a.contactnum AS contactnum,a.email AS
email,a.catalog_id AS catalog_id,a.consortia_id AS consortia_id,a.status AS status,a.approved_at AS
approved_at,a.approved_by AS approved_by,a.returned_at AS returned_at,a.created_at AS
created_at,a.updated_at AS updated_at,a.deleted_at AS deleted_at,b.title_statement AS
title_statement,date_format(a.created_at,'%M %d, %Y %T') AS request_at_string,if((a.status <> 'For
Approval'),if((a.returned_at = NULL),(to_days(cast(a.returned_at as date)) -
to_days(cast(a.approved_at as date))),(to_days(now()) - to_days(cast(a.approved_at as
date)))),NULL) AS total_days,date_format(a.approved_at,'%M %d, %Y %T') AS
approved_at_string,date_format(a.returned_at,'%M %d, %Y %T') AS returned_at_string from
(barrowed_books a left join holdings b on((b.catalog_id = a.catalog_id)))
```

view_holding_frontpages

```
select a.id AS id,a.catalog_id AS catalog_id,a.material_type_id AS
material_type_id,b.FrontPageLocation AS FrontPageLocation from (holdings a left join frontpages b
on((a.catalog_id = b.catalog_id))) where (b.FrontPageLocation is not null)
```

view_holdings

```
select a.id AS id,a.id AS catalog_id,a.circulation_number AS circulation_number,a.accession_number
AS accession_number,a.title_statement AS title_statement,if((a.summary is null),'N/A',a.summary) AS
summary,a.material_type_id AS material_type_id,a.series_statement_id AS
series_statement_id,a.catalog_date AS catalog_date,a.date_acquired AS date_acquired,a.Edition AS
edition,date_format(a.created_at,'%M %d, %Y %T') AS created_at,a.CreatedBy AS
CreatedBy,if((a.catalog_date = ''),'Uncataloged','Cataloged') AS catalog_status,b.name AS
material,c.description AS series_description,a.call_number AS call_number,a.publisher AS
publisher,a.pub_url AS pub_url,(select group_concat(substr(authors.author_name,8) separator ' - ')
from authors where (authors.catalog_id = a.id) group by authors.catalog_id) AS
joint_authors,a.general_note AS general_note,a.publication_date AS pub_year,a.deleted_at AS
deleted_at from ((holdings a left join materials b on((a.material_type_id = b.id))) left join
series c on((c.id = a.series_statement_id))) where (a.deleted_at is null)
```

view_holdings_uncatalog

```
select a.id AS id,a.catalog_id AS catalog_id,a.call_number AS call_number,a.catalog_date AS
catalog_date,a.CatalogSource AS CatalogSource,a.material_type_id AS material_type_id,a.ISBN AS
ISBN,a.ISSN AS ISSN,a.CallNumber AS CallNumber,a.CallNumberFull AS CallNumberFull,a.title_statement
AS title_statement,a.edition_statement AS edition_statement,a.edition_statement_remainder AS
edition_statement_remainder,a.PhysicalDescriptionEtc AS PhysicalDescriptionEtc,a.MediaTypeCode AS
MediaTypeCode,a.MediaSource AS MediaSource,a.CarrierSource AS CarrierSource,a.series_statement_id
AS series_statement_id,a.series_statement AS series_statement,a.general_note AS
general_note,a.bibliography AS bibliography,a.FormattedContents AS
FormattedContents,a.ScopeandContent AS ScopeandContent,a.summary AS
Summary,a.AdditionalPhysicalFormAvailableNote AS
AdditionalPhysicalFormAvailableNote,a.PublicationStatus AS PublicationStatus,a.IsActive AS
IsActive,a.deleted AS deleted,a.deleted_at AS deleted_at,a.CreatedBy AS CreatedBy,a.created_at AS
created_at,a.UpdatedBy AS UpdatedBy,a.updated_at AS updated_at,a.LanguageID AS LanguageID,a.Edition
AS Edition,a.VolumeNo AS VolumeNo,a.BroadClass AS BroadClass,a.ContentTypeID AS
ContentTypeID,a.ContentTypeSource AS ContentTypeSource,a.VolumeSequentialDesignation AS
VolumeSequentialDesignation,a.PhysicalFormAvailableNote AS
PhysicalFormAvailableNote,a.AvailabilitySource AS AvailabilitySource,a.AvailabilityConditions AS
AvailabilityConditions,a.MediaTypeID AS MediaTypeID,a.CarrierTypeID AS
CarrierTypeID,a.CarrierTypeCode AS CarrierTypeCode,a.ContentTypeCode AS
```

```
ContentTypeCode,a.DissertationNote AS DissertationNote,a.AbbreviatedTitle AS
AbbreviatedTitle,a.SoundCharacteristics AS SoundCharacteristics,a.DatesofPublication AS
DatesofPublication,a.TimePeriodofCreation AS TimePeriodofCreation,a.CitationReferencesNote AS
CitationReferencesNote,a.SpecialCodedDates AS
SpecialCodedDates,a.ProductionDistributionandYearSubmitted AS
ProductionDistributionandYearSubmitted,a.RestrictiononAccessNote AS
RestrictiononAccessNote,a.TypesofReportandPeriodCoveredNote AS
TypesofReportandPeriodCoveredNote,a.consortia_id AS consortia_id,a.accession_number AS
accession_number,a.circulation_number AS circulation_number,a.cost_price AS
cost_price,a.acquisition_modes AS acquisition_modes,a.copy_number AS
copy_number,a.classifications_system AS classifications_system,a.date_acquired AS
date_acquired,a.source AS source,a.user_restriction AS user_restriction,a.item_status AS
item_status,a.temporary_location AS temporary_location,a.nonpublic_note AS
nonpublic_note,a.frequency AS frequency,a.volume AS volume,a.issue_date AS
issue_date,a.issue_number AS issue_number,a.publication_date AS
publication_date,a.classification_number AS classification_number,a.author_number AS
author_number,a.copyright_date AS copyright_date,a.title_remainder AS
title_remainder,a.statement_responsibility AS statement_responsibility,a.extent AS
extent,a.other_physical AS other_physical,a.dimension AS dimension,a.imprint AS
imprint,a.type_of_imprint AS type_of_imprint,a.publication_place AS publication_place,a.publisher
AS publisher,a.copies AS copies,b.name AS material_desc,(select group_concat(authors.author_name2
separator ' - ') from authors where (authors.catalog_id = a.catalog_id) group by
authors.catalog_id) AS joint_authors from (holdings a left join materials b on((a.material_type_id
= b.id))) where ((a.catalog_date is null) and (a.deleted_at is not null))
```

view_inquiries

```
select a.id AS id,a.user_id AS user_id,a.fullname AS fullname,a.contactnum AS contactnum,a.email AS
email,a.catalog_id AS catalog_id,a.subject AS subject,a.message AS message,a.created_at AS
created_at,a.updated_at AS updated_at,b.title_statement AS
title_statement,date_format(a.created_at,'%M %d, %Y %T') AS created_at_string,(select
count(reply_inquiries.id) from reply_inquiries where (reply_inquiries.inquiry_id = a.id)) AS
total_msg,(select count(view_inquiry_threads.id) from view_inquiry_threads where
((view_inquiry_threads.id = a.id) and (view_inquiry_threads.seen is null) and
(view_inquiry_threads.replied_by <> a.user_id))) AS new_reply from (inquiries a left join holdings
b on((b.catalog_id = a.catalog_id)))
```

view_inquiry_threads

```
select b.id AS id,b.user_id AS user_id,a.user_id AS replied_by,c.name AS replied_by_name,a.message
AS replied_msg,a.seen AS seen,date_format(b.created_at,'%M %d, %Y %T') AS
created_at_string,date_format(a.created_at,'%M %d, %Y %T') AS thread_date from ((reply_inquiries a
left join inquiries b on((a.inquiry_id = b.id))) left join users c on((a.user_id = c.id)))
```

view_most_downloaded

```
select count(a.id) AS total,a.catalog_id AS catalog_id from (downloads a join catalogs c
on((a.catalog_id = c.id))) where (c.deleted_at is null) group by a.catalog_id order by count(a.id)
desc limit 0,10
```

view_most_viewed

```
select count(a.id) AS total,a.catalog_id AS catalog_id from (views a join catalogs c
on((a.catalog_id = c.id))) where (c.deleted_at is null) group by a.catalog_id order by count(a.id)
desc limit 0,5
```

view_ratings

```
select a.id AS id,a.catalog_id AS catalog_id,a.user_id AS user_id,a.rating AS rating,a.remarks AS
remarks,a.created_at AS created_at,a.updated_at AS updated_at,b.name AS name from (ratings a left
join users b on((b.id = a.user_id)))
```

view_searches

```
select a.id AS id,concat(a.title_statement,a.summary) AS search,a.material_type_id AS
material_type_id,b.name AS material from (holdings a left join materials b on((b.id =
a.material_type_id)))
```

view_to_reviews

```
select a.id AS id,a.catalog_id AS catalog_id,a.user_id AS user_id,a.to_review AS
to_review,a.created_at AS created_at,a.updated_at AS updated_at,b.title_statement AS
title_statement from (downloads a left join holdings b on((b.catalog_id = a.catalog_id)))
```

Appendix D Migration history

Table 14. Migrations in order of execution.

Migration	Effect
`2019_12_14_000001_create_personal_access_tokens_table`	Laravel Sanctum token table (framework default; unused).
`2025_09_15_114348_drop_all_tables`	Drops every table and view. Destructive; part of the baseline only.
`2025_09_15_114349_migration`	Baseline: executes `database/sql/elib_new.sql` (56 tables, 11 views).
`2026_07_12_000000_create_settings_table`	Key-value `settings` (guarded; the table is also in the dump).
`2026_07_13_000000_add_guest_replied_at_to_inquiries`	`inquiries.guest_replied_at`.
`2026_07_13_100000_add_ban_block_and_reservation_messages`	`users.is_banned`, `users.is_blocked`; `reservation_messages`.
`2026_07_14_000000_create_reads_table`	`reads` with UNIQUE(user_id, catalog_id).
`2026_07_14_200000_add_subject_to_inquiries`	`inquiries.subject`; recreates `view_inquiries`.
`2026_07_15_000000_create_inquiry_seens_table`	`inquiry_seens` with UNIQUE(inquiry_id, user_id).
`2026_07_18_000000_widen_accession_number`	Widens `accession_number` in `catalogs` and `view_catalogs`.
`2026_07_19_000000_create_rag_sync_queue_table`	The RAG outbox `rag_sync_queue`.
`2026_07_28_000000_add_user_agent_to_views_table`	`views.user_agent`.
`2026_10_07_000000_widen_title_columns`	Title columns to TEXT in `copies`, `view_catalogs`, `circulation_reservations`.
`2026_10_07_100000_create_material_type_maps_table`	`material_type_maps`; `catalog_imports.material_map`.
`2026_10_07_200000_material_types_utility`	`materials.is_active`, `is_marc_type`, audit columns, UNIQUE(name); permission rows.
`2026_10_08_000000_add_subject_to_catalog_imports`	`catalog_imports.subject`.
`2026_10_12_000000_add_search_fields_to_view_catalogs`	`view_catalogs.subjects`, `pub_year_num` (indexed); `saved_records` with foreign keys.
`2026_10_13_000000_settings_and_ai_sync_monitor_permissions`	Permission rows for Settings and the AI Sync Monitor.

Appendix E Reference vocabularies

Contents of the lookup tables that give codes their meaning (no personal data).

Table 15. Circulation states (circulation_status).

order_no	Name	Meaning
1	For Approval	The booking is being reviewed and validated.
2	Approved	Passed the checks; ready for check-out.
3	Open	Checked out and received by the patron.
4	Declined	Reservation request denied.
5	Closed	Returned and fines settled.

Table 16. Copy states (copies_status).

order_no	Name	Meaning
1	Available	Ready for reservation or borrowing.
2	Reserved	On hold for an approved reservation.
3	Borrowed	Checked out until returned.
4	Not Available	Marked unusable by staff.
5	Deleted	Soft-deleted.
6	Lost	Lost.
7	For Repair	Damaged; needs repair.

Table 17. Other vocabularies.

Table	Values
`patrontypes`	Business/Private Sector, Farmer, Government Employee, Policymaker, Professional, Researcher, Student, Others
`circulation_fines`	Water Damage, Cover Damage, Spine Damage, Page Tears, Highlighting and Underlining, Writing or Annotations, Dog-Eared Pages, Loose Binding, Dust Jacket Damage, Excessive Wear
`marc_class_sys`	LOC (Library of Congress), DDC (Dewey Decimal Classification)
`imprints`	Not Applicable/Not Provided/Earliest, Intervening, Current/Last
`type_of_imprints`	Production, Publication
`roles`	super_admin, serial_admin, cataloging_admin, circulation_admin, faculty, system_admin, developer
`consortias`	CorCAARRD, MAARRDEC, WESMAARRDEC, ILAARRDEC, BCAARRD, NOMCAARRD, CVAARRD, WESVAARRDEC, SMAARRDEC, CLAARRDEC, CVAARRDEC, SOXAARRDEC, STAARRDEC, ViCARP, CCAARRD, PCAARRD
`settings` keys	`ai_mode_enabled`, `enable_placeholder_covers`, `seo_discourage_indexing`, `seo_verification_google`

Appendix F Reproducing the evidence

The queries below regenerate the evidence. They are read-only and return structure or counts only. Run Q1 against both `elibrarydb_test` (after `vendor/bin/phpunit` has built it from the migrations) and `elibrarydb`, then compare.

Q1 Schema introspection

```
SELECT TABLE_NAME, TABLE_TYPE, ENGINE, TABLE_COLLATION FROM information_schema.TABLES WHERE TABLE_SCHEMA = DATABASE();
SELECT TABLE_NAME, ORDINAL_POSITION, COLUMN_NAME, COLUMN_TYPE, IS_NULLABLE, COLUMN_DEFAULT, COLUMN_KEY, EXTRA FROM information_schema.COLUMNS WHERE TABLE_SCHEMA = DATABASE();
SELECT TABLE_NAME, INDEX_NAME, NON_UNIQUE, SEQ_IN_INDEX, COLUMN_NAME FROM information_schema.STATISTICS WHERE TABLE_SCHEMA = DATABASE();
SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA = DATABASE() AND REFERENCED_TABLE_NAME IS NOT NULL;
SELECT TABLE_NAME, VIEW_DEFINITION FROM information_schema.VIEWS WHERE TABLE_SCHEMA = DATABASE();
```

Q2 Orphaned implicit references

For each implicit reference child.c → parent.p (30 were checked), count child rows whose value has no parent:

```
SELECT COUNT(*) FROM child c WHERE c.c IS NOT NULL AND NOT EXISTS (SELECT 1 FROM parent p WHERE p.p = c.c);
```

Table 18. Results of Q2 on 15 October 2026.

Reference	Rows	Orphans
`view_catalogs.catalog_id → catalogs.id`	34	0
`copies.catalog_id → catalogs.id`	0	0
`multimedia.catalog_id → catalogs.id`	4	0
`frontpages.catalog_id → catalogs.id`	34	0
`views.catalog_id → catalogs.id`	79	0
`downloads.catalog_id → catalogs.id`	6	0
`reads.catalog_id → catalogs.id`	2	0
`ratings.catalog_id → catalogs.id`	4	0
`inquiries.catalog_id → catalogs.id`	0	0
`circulations.catalog_id → catalogs.id`	0	0
`rag_sync_queue.catalog_id → catalogs.id`	34	0
`saved_records.catalog_id → catalogs.id`	11	0
`catalogs.material_type_id → materials.id`	34	0
`catalogs.catalog_import_id → catalog_imports.id`	34	0
`views.user_id → users.id`	59	0
`downloads.user_id → users.id`	6	0
`reads.user_id → users.id`	2	0

Reference	Rows	Orphans
ratings.user_id → users.id	4	0
circulations.user_id → users.id	0	0
inquiries.user_id → users.id	0	0
reply_inquiries.user_id → users.id	0	0
reply_inquiries.inquiry_id → inquiries.id	0	0
inquiry_seens.inquiry_id → inquiries.id	0	0
saved_records.user_id → users.id	11	0
csfs.user_id → users.id	0	0
model_has_roles.model_id → users.id	3	0
users.patrontypes_id → patrontypes.id	5	2
users.region_id → regions.id	3	0
circulation_fine_history.circulation_fine_id → circulation_fines.id	0	0
marctag_subfields_loc.marctags_loc_id → marctags_loc.id	2,606	0

Q3 Projection consistency

```
SELECT COUNT(*) FROM catalogs c WHERE NOT EXISTS (SELECT 1 FROM view_catalogs v WHERE
v.catalog_id = c.id);
SELECT COUNT(*) FROM catalogs c JOIN view_catalogs v ON v.catalog_id = c.id WHERE
v.is_published <> c.is_published OR (c.deleted_at IS NULL) <> (v.deleted_at IS NULL);
```

Results: 0 records without a projection; 0 publication and 0 deletion mismatches; 0 duplicate reviews; 0 records without MARC data.